

FILE: src/App.tsx

```
import './index.css';

import type { ReactElement } from 'react';
import { Routes, Route, Navigate } from 'react-router-dom';

import { ThemeProvider } from './context/ThemeContext';
import { AuthProvider } from './context/AuthContext';
import { useAuth } from './hooks/useAuth';
import Navbar from './Components/Navbar';
import Home from './pages/Home';
import Dashboard from './pages/Dashboard';
import Trade from './pages/Trade';
import Stake from './pages/Stake';
import WalletDashboard from './pages/WalletDashboard';
import Leaderboard from './pages/Leaderboard';
import Account from './pages/Account';
import Onboarding from './pages/Onboarding';

const ProtectedRoute = ({ children }: { children: ReactElement }) => {
  const { user, loading } = useAuth();
  if (loading) {
    return <div className="px-8 py-6">Checking session...</div>;
  }
  if (!user) {
    return <Navigate to="/Account" replace />;
  }
  return children;
};

function App() {
  return (
    <ThemeProvider>
      <AuthProvider>
        <>
          <Navbar />
          <main className="mt-10">
            <Routes>
              <Route path="/" element={<Home />} />
              <Route
                path="/Dashboard"
                element={
                  <ProtectedRoute>
                    <Dashboard />
                  </ProtectedRoute>
                }
              />
              <Route
                path="/Trade"
                element={
                  <ProtectedRoute>
                    <Trade />
                  </ProtectedRoute>
                }
              />
              <Route
                path="/Stake"
                element={
                  <ProtectedRoute>
                    <Stake />
                  </ProtectedRoute>
                }
              />
              <Route
                path="/Wallet-Dashboard"
                element={
                  <ProtectedRoute>
                    <WalletDashboard />
                  </ProtectedRoute>
                }
              />
            </Routes>
          </main>
        </>
      </AuthProvider>
    </ThemeProvider>
  );
}
```

```
    }
  />
  <Route
    path="/Leaderboard"
    element={
      <ProtectedRoute>
        <Leaderboard />
      </ProtectedRoute>
    }
  />
  <Route
    path = "/Onboarding"
    element = {
      <ProtectedRoute>
        <Onboarding/>
      </ProtectedRoute>
    }
  />
  <Route path="/Account" element={<Account />} />
  <Route path="*" element={<Navigate to="/" replace />} />
</Routes>
</main>
</>
</AuthProvider>
</ThemeProvider>
);
}

export default App;
```

FILE: src/Components/Button.tsx

```
import type buttonProps from '../types/Button';
//Retrieve button props from the buttonProps interface

const Button: React.FC<buttonProps> = ({children, variant, className, size, onClick, type, disabled}) => {
  //Creates a forward component so that the children can be passed like a regular button: <Button>
  text</Button>.

  //Receives button props - text, variant, className, size
  return (
    <button onClick = {onClick} className={`btn btn-${variant} btn-${size} ${className} `} type = {type}
disabled={disabled}>
      {children}
    </button>

  );
};

export default Button;
```

FILE: src/Components/CryptoChart.tsx

```

import { useState } from "react";
import SearchBar from "../SearchBar";
import { TradingViewChart } from "../TradingChart";
import { coinNameToTicker } from "../data/tickers";
import { useFetchCoinMetrics } from "../utils/FetchCoinMetrics";

const CryptoChart = () => {
  const [coin, setCoin] = useState("bitcoin");

  const handleSearch = (value: string) => {
    setCoin(value.toLowerCase());
  };

  const logoToken = import.meta.env.VITE_LOGO_API;
  const ticker = coinNameToTicker[coin];

  const { data, loading, error } = useFetchCoinMetrics(coin);

  // --- ERROR STATE: Unknown Coin Ticker ---
  if (!ticker) {
    return (
      <div className="p-8 text-center bg-surface min-h-screen text-primary">
        <SearchBar Search={handleSearch} />
        <p className="mt-8 text-xl text-danger">
          ⚠️ Unknown cryptocurrency: <strong>{coin}</strong>
        </p>
        <p className="text-muted">Try a common coin like "bitcoin" or "ethereum".</p>
      </div>
    );
  }

  // --- LOADING/ERROR STATES ---
  if (loading) {
    return (
      <div className="p-8 text-center bg-surface min-h-screen text-primary">
        <SearchBar Search={handleSearch} />
        <div className="mt-20 text-xl animate-pulse">Loading data for {coin}...</div>
      </div>
    );
  }

  if (error || !data) {
    return (
      <div className="p-8 text-center bg-surface min-h-screen text-primary">
        <SearchBar Search={handleSearch} />
        <div className="mt-20 text-xl text-danger">
          ❌ Fetch failed: {error || 'No data returned from API.'}
        </div>
      </div>
    );
  }

  // --- RENDER DASHBOARD ---

  // Determine color for 24h change (Success/Danger is common in trading UI)
  const changeColor = data.change24h >= 0 ? 'text-success' : 'text-danger';
  const changeSign = data.change24h >= 0 ? '↑' : '↓';

  return (
    <div className="min-h-screen bg-surface text-primary p-4 sm:p-8 font-sans">
      <div className="max-w-7xl mx-auto">

        { /* 1. Search Bar (Top Control) - Assuming SearchBar has minimal styling */ }
        <div className="mb-8">
          <SearchBar Search={handleSearch} />
        </div>

        { /* 2. Header and Price Metrics */ }
        <div className="flex items-center mb-6 space-x-4 border-b border-divider pb-4">

```

```

<img
  src={`https://img.logokit.com/crypto/${ticker}?token=${logoToken}`}
  alt={` ${coin} logo`}
  className="w-10 h-10 rounded-full shadow-md"
/>
<div>
  <h1 className="text-2xl font-semibold capitalize">{coin}</h1>
  <p className="text-sm text-muted">{ticker} / USD</p>
</div>

{/* Current Price and 24h Change */}
<div className="ml-auto flex items-end space-x-4">
  <p className="text-4xl font-bold">${data.price.toLocaleString('en-US', { minimumFractionDigits: 2,
maximumFractionDigits: 8 })}</p>
  <p className={`text-lg font-medium ${changeColor} mb-0.5`}>
    {changeSign} {data.change24h.toFixed(2)}%
  </p>
</div>
</div>

{/* 3. Main Chart (Placeholder) */}
<div className="bg-card rounded-lg p-2 shadow-lg h-[450px] mb-8 border border-divider">
  <TradingViewChart coin={coin} mode="candlestick" />
</div>

{/* 4. Key Metrics Grid */}
<div className="grid grid-cols-2 md:grid-cols-4 gap-4">
  {/* Market Cap */}
  <MetricCard
    title="Market Cap"
    value={`$${(data.marketCap / 1e9).toFixed(2)}B`}
    tooltip={`$${data.marketCap.toLocaleString()}`}
  />

  {/* 24h Volume */}
  <MetricCard
    title="24h Volume"
    value={`$${(data.volume24h / 1e9).toFixed(2)}B`}
    tooltip={`$${data.volume24h.toLocaleString()}`}
  />

  {/* Fully Diluted Valuation (FDV) */}
  <MetricCard
    title="FDV"
    value={data.fullyDilutedValuation
      ? `$${(data.fullyDilutedValuation / 1e9).toFixed(2)}B`
      : 'N/A'
    }
    tooltip={data.fullyDilutedValuation
      ? `$${data.fullyDilutedValuation.toLocaleString()}`
      : 'Not available'
    }
  />

  {/* Circulating Supply */}
  <MetricCard
    title="Circulating Supply"
    value={`$${(data.circulatingSupply / 1e6).toFixed(2)}M`}
    tooltip={data.circulatingSupply.toLocaleString()}
  />
</div>

{/* 5. Additional Supply Data (Muted Footer) */}
<div className="mt-8 text-xs text-muted text-right">
  Total Supply: {data.totalSupply?.toLocaleString() || 'Unlimited/N/A'}
</div>
</div>
</div>
);
};

```

```
// Helper component for clean metric display
const MetricCard = ({ title, value, tooltip }: { title: string, value: string, tooltip: string }) => (
  <div className="bg-card p-4 rounded-md shadow-sm border border-divider hover:shadow-lg transition duration-150 relative group">
    <p className="text-sm font-medium text-muted">{title}</p>
    <p className="text-xl font-semibold mt-1">{value}</p>
    { /* Tooltip implementation remains the same for functionality */ }
    <span className="absolute bottom-full left-1/2 transform -translate-x-1/2 mb-2 px-3 py-1 bg-tooltip text-xs text-tooltip-text rounded opacity-0 group-hover:opacity-100 transition-opacity duration-300 pointer-events-none whitespace-nowrap">
      {tooltip}
    </span>
  </div>
);

export default CryptoChart;
```

FILE: src/Components/DMLMToggle.tsx

```
import { useTheme } from "../hooks/useTheme";
import Button from "../Button";
export const Toggle = () => {
  const {isDark, toggleTheme} = useTheme();
  return (
    <Button
      onClick={toggleTheme}
      size="lg"
      type="button"
      variant="secondary"
    >
      {isDark ? <img className = "w-5 h-5" src = "/public/sun.svg"/> : <img className = "w-5 h-5" src =
"/public/moon.svg"/>}
    </Button>
  );
};
```

FILE: src/Components/Navbar.tsx

```

import Button from "../Button";
import { useNavigate, useLocation } from "react-router-dom";
import { Links } from "../lib/links";
import { Toggle } from "../DMLMToggle";

const Navbar = () => {
  const navigate = useNavigate();
  const location = useLocation();
  const { pathname } = location;
  return (

<nav className="bg-neutral-primary w-full z-20 top-0 start-0 ">
  <div className="max-w-7xl flex flex-wrap items-center justify-between mx-auto p-4">
    <a href="/" className="flex items-center space-x-3 rtl:space-x-reverse">

      <span className="self-center text-xl text-heading font-semibold whitespace-nowrap">Block Finance</span>
    </a>
    <button data-collapse-toggle="navbar-default" type="button" className="inline-flex items-center p-2 w-10 h-10 justify-center text-sm text-body rounded-base md:hidden hover:bg-neutral-secondary-soft hover:text-heading focus:outline-none focus:ring-2 focus:ring-neutral-tertiary" aria-controls="navbar-default" aria-expanded="false">
      <span className="sr-only">Open main menu</span>
      <svg className="w-6 h-6" aria-hidden="true" xmlns="http://www.w3.org/2000/svg" width="24" height="24" fill="none" viewBox="0 0 24 24"><path stroke="currentColor" strokeLinecap="round" strokeWidth="2" d="M5 7h14M5 12h14M5 17h14"/></svg>
    </button>
    <div className="hidden w-full md:block md:w-auto" id="navbar-default">
      <ul className="font-medium flex flex-col p-4 md:p-0 mt-4 border rounded-lg md:flex-row md:space-x-8 md:mt-0 md:border-0">

        {Links.map((item) => (
          <li key={item.link}>
            <Button
              onClick={() => navigate(item.link)}
              type="button"
              variant="secondary"
              size="md"
              //Created a template string that checked whether the pathname is equal to the link and changed the
              color of the text accordingly.
              className={` ${pathname === item.link ? 'text-gray-800' : ''} `}
            >
              {item.name}
            </Button>
          </li>
        ))}
      </ul>
    </div>
  </div>
</nav>

  );
};
export default Navbar

```

FILE: src/Components/NewsCard.tsx

```

interface NewsCardProps {
  title: string;
  description: string | null;
  link: string;
  imageUrl: string | null;
  pubDate: string;
  source: string;
}

const NewsCard = ({ title, description, link, imageUrl, pubDate, source }: NewsCardProps) => {
  const formatDate = (dateString: string) => {
    const date = new Date(dateString);
    const now = new Date();
    //Difference in hours
    const diffInHours = Math.floor((now.getTime() - date.getTime()) / (1000 * 60 * 60));
    //Use if statements to determine how recent a post is for the user - improving user experience
    if (diffInHours < 1) return 'Just now';
    if (diffInHours < 24) return `${diffInHours}h ago`;
    const diffInDays = Math.floor(diffInHours / 24);
    if (diffInDays < 7) return `${diffInDays}d ago`;
    return date.toLocaleDateString('en-US', { month: 'short', day: 'numeric' });
  };

  return (
    <a
      href={link}
      target="_blank"
      rel="noopener noreferrer"
      className="
        block rounded-xl border border-[var(--border-color)]
        bg-[var(--surface-color)] text-[var(--text-color)]
        no-underline overflow-hidden hover:shadow-md
        transition-shadow duration-200 group
      "
    >

      <div className="w-full h-40 overflow-hidden">
        <img
          //If image url isn't present then use the backup image
          src={imageUrl || "/backup.webp"}
          alt={title}
          //Typical tailwindcss animation for image upscale on hover
          className="w-full h-full object-cover group-hover:scale-105 transition-transform duration-200"
        />
      </div>

      <div className="p-4">
        <div className="flex items-center justify-between mb-2">
          <span className="text-xs font-medium uppercase text-[var(--muted-text-color)]">
            {source}
          </span>
          <span className="text-xs text-[var(--muted-text-color)]">
            {formatDate(pubDate)}
          </span>
        </div>
        <h3 className="text-sm font-semibold mb-2 line-clamp-2">
          {title}
        </h3>
        {/*If description then showcase it */}
        {description && (
          <p className="text-xs text-[var(--muted-text-color)] line-clamp-2">
            {description}
          </p>
        )}
      </div>
    </a>
  );
};

```

```
export default NewsCard;
```

FILE: src/Components/SearchBar.tsx

```

import { useState } from 'react';
import Button from './Button';

interface SearchBarProps {
  Search: (coin: string) => void;
}

const SearchBar: React.FC<SearchBarProps> = ({ Search }) => {
  const [query, setQuery] = useState("");

  const handleClick = () => {
    Search(query.trim());
  };

  return (
    <>

      <form
        className="max-w-md mx-auto"
        onSubmit={(e) => {
          e.preventDefault(); // prevents page reload
          handleClick();
        }}
      >
        <label htmlFor="search" className="block mb-2.5 text-sm font-medium text-heading sr-only">
          Search
        </label>

        <div className="relative">
          <div className="absolute inset-y-0 start-0 flex items-center ps-3 pointer-events-none">
            <svg
              className="w-4 h-4 text-body"
              aria-hidden="true"
              xmlns="http://www.w3.org/2000/svg"
              width="24"
              height="24"
              fill="none"
              viewBox="0 0 24 24"
            >
              <path stroke="currentColor" strokeLinecap="round" strokeWidth="2" d="m21 21-3.5-3.5M17 10a7 7 0 1
1-14 0 7 7 0 0 1 14 0Z" />
            </svg>
          </div>

          <input
            type="search"
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            className="block w-full p-3 ps-9 bg-neutral-secondary-medium text-heading text-sm rounded-base
shadow-xs placeholder:text-body"
            placeholder="Search"
          />

          <Button
            variant = "secondary"
            size = "sm"
            type="submit"
            className="absolute end-0 bottom-0 border border-transparent focus:ring-4 focus:ring-brand-medium
shadow-xs font-medium leading-5 rounded text-xs px-3 py-1.5"
          >
            Search
          </Button>
        </div>
      </form></>

    );
  };
}

export default SearchBar;

```

FILE: src/Components/Tooltip.tsx

```

import { HelpCircle } from 'lucide-react';
import { useState, type ReactNode } from 'react';

interface TooltipProps {
  text: string;
  children?: ReactNode;
  position?: 'top' | 'bottom';
}

const Tooltip = ({ text, children, position = 'top' }: TooltipProps) => {
  const [open, setOpen] = useState(false);
  const tooltipPositionClass = position === 'top' ? 'bottom-full mb-2' : 'top-full mt-2';

  const accessibleLabel =
    typeof children === 'string' && children.trim() !== '' ? `More info about ${children}` : 'More info';

  return (
    <span className="relative inline-flex align-baseline">
      <span
        role="button"
        tabIndex={0}
        className="inline-flex items-baseline cursor-help border-b border-dashed border-current bg-transparent
p-0"
        onMouseEnter={() => setOpen(true)}
        onMouseLeave={() => setOpen(false)}
        onFocus={() => setOpen(true)}
        onBlur={() => setOpen(false)}
        onClick={() => setOpen((value) => !value)}
        onKeyDown={(event) => {
          if (event.key === 'Enter' || event.key === ' ') {
            event.preventDefault();
            setOpen((value) => !value);
          }
        }}
        style={{ lineHeight: 'inherit' }}
        aria-label={accessibleLabel}
        aria-expanded={open}
      >
        <span className="inline">{children}</span>
        <HelpCircle className="ml-1 inline-block h-3 w-3" style={{ verticalAlign: 'baseline', position:
'relative', top: '0.05em' }} />
      </span>

      {open && (
        <span
          role="tooltip"
          className={`pointer-events-none absolute left-1/2 z-[70] w-max max-w-xs -translate-x-1/2 rounded-lg
border border-[var(--border-color)] bg-[var(--surface-color)] px-3 py-2 text-xs shadow-xl
${tooltipPositionClass}`}
          style={{ color: 'var(--text-color)' }}
        >
          {text}
        </span>
      )}
    </span>
  );
};

export default Tooltip;

```

FILE: src/Components/TradingChart.tsx

```

import {
  createChart,
  ColorType,
  CandlestickSeries,
  LineSeries,
  type CandlestickData,
  type IChartApi,
  type ISeriesApi,
  type UTCTimestamp,
} from 'lightweight-charts';
import { useEffect, useRef, useState } from 'react';
import { fetchCandleData } from '../utils/FetchCandleData';
import { useTheme } from '../hooks/useTheme';

type ChartMode = 'line' | 'candlestick';

interface TradingViewChartProps {
  coin: string;
  mode?: ChartMode;
  onMarketDataChange?: (marketData: {
    lastPrice: number | null;
    change24h: number | null;
  }) => void;
}

const TradingViewChart = ({
  coin,
  mode = 'candlestick',
  onMarketDataChange,
}: TradingViewChartProps) => {
  const { isDark } = useTheme();
  const chartContainerRef = useRef<HTMLDivElement | null>(null);
  const chartRef = useRef<IChartApi | null>(null);
  const candleSeriesRef = useRef<ISeriesApi<'Candlestick'> | null>(null);
  const lineSeriesRef = useRef<ISeriesApi<'Line'> | null>(null);
  const [candles, setCandles] = useState<CandlestickData<UTCTimestamp>[]>([]);

  useEffect(() => {
    if (!chartContainerRef.current) {
      return;
    }

    const chart = createChart(chartContainerRef.current, {
      layout: {
        background: { type: ColorType.Solid, color: isDark ? '#000000' : '#ffffff' },
        textColor: isDark ? '#e5e7eb' : '#1f1f1f',
      },
      grid: {
        vertLines: { color: isDark ? '#34343a' : '#e5e7eb' },
        horzLines: { color: isDark ? '#34343a' : '#e5e7eb' },
      },
      width: chartContainerRef.current.clientWidth,
      height: 400,
    });

    if (mode === 'candlestick') {
      candleSeriesRef.current = chart.addSeries(CandlestickSeries, {
        upColor: '#26a69a',
        downColor: '#ef5350',
        wickUpColor: '#26a69a',
        wickDownColor: '#ef5350',
        borderVisible: false,
      });
      lineSeriesRef.current = null;
    } else {
      lineSeriesRef.current = chart.addSeries(LineSeries, {
        color: '#2962FF',
        lineWidth: 2,
      });
    }
  });

```

```

    candleSeriesRef.current = null;
  }

  chartRef.current = chart;

  const handleResize = () => {
    if (chartContainerRef.current) {
      chart.applyOptions({ width: chartContainerRef.current.clientWidth });
    }
  };

  window.addEventListener('resize', handleResize);

  return () => {
    window.removeEventListener('resize', handleResize);
    chart.remove();
    chartRef.current = null;
    candleSeriesRef.current = null;
    lineSeriesRef.current = null;
  };
}, [isDark, mode]);

useEffect(() => {
  let mounted = true;

  const loadCandles = async () => {
    setCandles([]);
    const nextCandles = await fetchCandleData(coin);
    if (mounted) {
      setCandles(nextCandles);
    }
  };

  void loadCandles();

  return () => {
    mounted = false;
  };
}, [coin]);

useEffect(() => {
  if (candles.length === 0) {
    return;
  }

  if (mode === 'candlestick' && candleSeriesRef.current) {
    candleSeriesRef.current.setData(candles);
    chartRef.current?.timeScale().fitContent();
    return;
  }

  if (mode === 'line' && lineSeriesRef.current) {
    const lineData = candles.map((candle) => ({
      time: candle.time,
      value: candle.close,
    }));
    lineSeriesRef.current.setData(lineData);
    chartRef.current?.timeScale().fitContent();
  }
}, [candles, mode]);

const latestChartPrice = candles.length > 0 ? candles[candles.length - 1].close : null;
const price24hAgo = candles.length > 24 ? candles[candles.length - 25].close : null;
const chartChange24h =
  latestChartPrice !== null &&
  price24hAgo !== null &&
  Number.isFinite(price24hAgo) &&
  price24hAgo > 0
  ? ((latestChartPrice - price24hAgo) / price24hAgo) * 100
  : null;

```

```
useEffect(() => {
  if (!onMarketDataChange) {
    return;
  }

  onMarketDataChange({
    lastPrice: latestChartPrice,
    change24h: chartChange24h,
  });
}, [chartChange24h, latestChartPrice, onMarketDataChange]);

return <div ref={chartContainerRef} style={{ width: '100%', height: '400px' }} />;
};

export type { ChartMode };
export { TradingViewChart };
```

FILE: src/context/AuthContext.tsx

```

import {
  useEffect,
  useState,
  type ReactNode,
} from 'react';
import { supabase } from '../lib/supabaseClient';
import { AuthContext, type AuthContextType } from './authContextStore';

export const AuthProvider = ({ children }: { children: ReactNode }) => {
  const [user, setUser] = useState<AuthContextType['user']>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const init = async () => {
      const { data, error } = await supabase.auth.getUser();
      if (!error && data.user) {
        setUser(data.user);
      }
      setLoading(false);
    };

    void init();

    const {
      data: { subscription },
    } = supabase.auth.onAuthStateChange((_event, session) => {
      setUser(session?.user ?? null);
    });

    return () => {
      subscription.unsubscribe();
    };
  }, []);

  // 📁 LOGIN (email + password)
  const signIn: AuthContextType['signIn'] = async (email, password) => {
    const { error } = await supabase.auth.signInWithPassword({
      email,
      password,
    });

    if (error) {
      return { error: error.message };
    }
    return {};
  };

  // 🆕 SIGNUP (email + password + username)
  const signUp: AuthContextType['signUp'] = async (
    email,
    password,
    username
  ) => {
    const { error } = await supabase.auth.signUp({
      email,
      password,
      options: {
        data: {
          username, // The username is in the user_metadata field
        },
      },
    });

    if (error) {
      return { error: error.message };
    }
    return {};
  };
};

```

```
const signOut = async () => {  
  await supabase.auth.signOut();  
};  
  
return (  
  <AuthContext.Provider value={{ user, loading, signIn, signUp, signOut }}>  
    {children}  
  </AuthContext.Provider>  
);  
};
```

FILE: src/context/ThemeContext.tsx

```
import { useEffect, type ReactNode } from "react";
import useLocalStorage from "use-local-storage";
import { ThemeContext } from "../hooks/useTheme"

export const ThemeProvider = ({ children }: { children: ReactNode }) => {
  const preference = window.matchMedia("(prefers-color-scheme: dark)").matches;
  const [isDark, setIsDark] = useLocalStorage("isDark", preference);

  useEffect(() => {
    const root = document.documentElement; // Fetches the root styling element
    if (isDark) {
      root.setAttribute("data-theme", "dark"); // Sets the data-theme to dark
    } else {
      root.setAttribute("data-theme", "light"); // Sets the data-theme to light
    }
    //Each time the isDark variable changes the useEffect is triggered
  }, [isDark]);
  const toggleTheme = () => {
    setIsDark(!isDark);
  };

  return (
    <ThemeContext.Provider value={{ isDark, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};
```

FILE: src/context/authContextStore.ts

```
import { createContext } from 'react';
import type { User } from '@supabase/supabase-js';

export type AuthContextType = {
  user: User | null;
  loading: boolean;
  signIn: (email: string, password: string) => Promise<{ error?: string }>;
  signUp: (
    email: string,
    password: string,
    username: string
  ) => Promise<{ error?: string }>;
  signOut: () => Promise<void>;
};

export const AuthContext = createContext<AuthContextType | undefined>(undefined);
```

FILE: src/data/BadgeDefinitions.ts

```
import { type BadgeDefinition } from "../types/Badge";
export const badgeDefinitions: BadgeDefinition[] = [
  {
    id: 'first_trade',
    symbol: '↗',
    title: 'First Fill',
    description: 'Place your first trade.',
    isUnlocked: (_xp, tradeCount) => tradeCount >= 1,
  },
  {
    id: 'xp_200',
    symbol: '🎯',
    title: '200 XP',
    description: 'Reach 200 total XP.',
    isUnlocked: (xp) => xp >= 200,
  },
  {
    id: 'ten_trades',
    symbol: '📈',
    title: 'Active Trader',
    description: 'Complete 10 trades.',
    isUnlocked: (_xp, tradeCount) => tradeCount >= 10,
  },
  {
    id: 'xp_500',
    symbol: '🏆',
    title: '500 XP',
    description: 'Reach 500 total XP.',
    isUnlocked: (xp) => xp >= 500,
  },
  {
    id: 'xp_1000',
    symbol: '👑',
    title: '1,000 XP',
    description: 'Reach 1,000 total XP.',
    isUnlocked: (xp) => xp >= 1000,
  },
];
```

FILE: src/data/stakeProviders/avalanche.ts

```
import type { StakeProvider } from '../../types/stakeProvider';

const avalancheProviders: StakeProvider[] = [
  { id: 'glacier-avax', name: 'Glacier', commission: 6, uptimeModifier: 1.01, trustScore: 84, apy: 9.6 },
  { id: 'summit-avax', name: 'Summit', commission: 5, uptimeModifier: 1.03, trustScore: 88, apy: 10.2 },
  { id: 'vector-avax', name: 'Vector', commission: 8, uptimeModifier: 0.99, trustScore: 82, apy: 8.9 },
];

export default avalancheProviders;
```

FILE: src/data/stakeProviders/cardano.ts

```
import type { StakeProvider } from '../../types/stakeProvider';

const cardanoProviders: StakeProvider[] = [
  { id: 'atlas-ada', name: 'Atlas Pool', commission: 3, uptimeModifier: 1.01, trustScore: 87, apy: 5.4 },
  { id: 'lido-ada', name: 'Lido ADA', commission: 4, uptimeModifier: 1.0, trustScore: 85, apy: 5.0 },
  { id: 'orio-ada', name: 'Orio', commission: 2, uptimeModifier: 1.03, trustScore: 90, apy: 5.8 },
];

export default cardanoProviders;
```

FILE: src/data/stakeProviders/ethereum.ts

```
import type { StakeProvider } from '../../types/stakeProvider';

const ethereumProviders: StakeProvider[] = [
  { id: 'beacon-eth', name: 'Beacon Prime', commission: 12, uptimeModifier: 1.0, trustScore: 90, apy: 4.7 },
  { id: 'lighthouse-eth', name: 'Lighthouse', commission: 10, uptimeModifier: 1.01, trustScore: 88, apy: 4.9 },
  { id: 'altair-eth', name: 'Altair', commission: 14, uptimeModifier: 0.98, trustScore: 85, apy: 4.4 },
];

export default ethereumProviders;
```

FILE: src/data/stakeProviders/index.ts

```
import type { StakeProvider } from '../types/stakeProvider';
import avalancheProviders from './avalanche';
import cardanoProviders from './cardano';
import ethereumProviders from './ethereum';
import polkadotProviders from './polkadot';
import solanaProviders from './solana';

const stakeProvidersByChain: Record<string, StakeProvider[]> = {
  Solana: solanaProviders,
  Polkadot: polkadotProviders,
  Cardano: cardanoProviders,
  Avalanche: avalancheProviders,
  Ethereum: ethereumProviders,
};

export default stakeProvidersByChain;
```

FILE: src/data/stakeProviders/polkadot.ts

```
import type { StakeProvider } from '../../types/stakeProvider';

const polkadotProviders: StakeProvider[] = [
  { id: 'nova-dot', name: 'Nova Pool', commission: 9, uptimeModifier: 1.01, trustScore: 86, apy: 12.3 },
  { id: 'parity-dot', name: 'Parity One', commission: 7, uptimeModifier: 1.02, trustScore: 89, apy: 13.1 },
  { id: 'chainops-dot', name: 'ChainOps', commission: 10, uptimeModifier: 0.97, trustScore: 80, apy: 11.4 },
];

export default polkadotProviders;
```

FILE: src/data/stakeProviders/solana.ts

```
import type { StakeProvider } from '../types/stakeProvider';

const solanaProviders: StakeProvider[] = [
  { id: 'blaze-sol', name: 'Blaze Stake', commission: 6, uptimeModifier: 1.02, trustScore: 88, apy: 10.0 },
  { id: 'kraken-sol', name: 'Kraken', commission: 8, uptimeModifier: 1.0, trustScore: 84, apy: 9.0 },
  { id: 'phantom-sol', name: 'Phantom', commission: 5, uptimeModifier: 1.04, trustScore: 90, apy: 12.0 },
  { id: 'cronos-sol', name: 'Cronos', commission: 10, uptimeModifier: 0.98, trustScore: 78, apy: 10.0 },
  { id: 'axiom-sol', name: 'Axiom', commission: 7, uptimeModifier: 1.05, trustScore: 92, apy: 15.0 },
];

export default solanaProviders;
```

FILE: src/data/tickers.ts

```
export const coinNameToTicker: Record<string, string> = {
  bitcoin: 'BTC',
  ethereum: 'ETH',
  tether: 'USDT',
  'binance coin': 'BNB',
  solana: 'SOL',
  'usd coin': 'USDC',
  xrp: 'XRP',
  cardano: 'ADA',
  dogecoin: 'DOGE',
  avalanche: 'AVAX',
  tron: 'TRX',
  polkadot: 'DOT',
  polygon: 'MATIC',
  litecoin: 'LTC',
  chainlink: 'LINK',
  stellar: 'XLM',
  'bitcoin cash': 'BCH',
  cosmos: 'ATOM',
  monero: 'XMR',
  uniswap: 'UNI',
  vechain: 'VET',
  aave: 'AAVE',
  maker: 'MKR',
  'wrapped bitcoin': 'WBTC',
};

// Simple list of major trading pairs used by the Trade page.
// This is intentionally small and static – it is ONLY for the simulator UI,
// not for driving live market data.
```

FILE: src/hooks/useAuth.ts

```
import { useContext } from 'react';
import { AuthContext } from '../context/authContextStore';

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return ctx;
}

export default useAuth;
```

FILE: src/hooks/useTheme.ts

```
import {useContext, createContext} from "react";
type ThemeContextType = {
  isDark: boolean;
  toggleTheme: () => void;
};
export const ThemeContext = createContext<ThemeContextType | undefined>(undefined);
export const useTheme = () => {
  const context = useContext(ThemeContext);
  if (context === undefined) {
    throw new Error("useTheme must be used within a ThemeProvider");
  }
  return context;
};
```

FILE: src/index.css

```

@import "tailwindcss";

:root {
  font-family: 'Inter', system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI',
    Roboto, Helvetica, Arial, sans-serif;
  line-height: 1.5;
  font-weight: 400;
  color-scheme: light;

  --background-color: #f4f4f5;
  --surface-color: #ffffff;
  --muted-surface-color: #eeece8;
  --border-color: #d6d3d1;
  --primary-text-color: #1c1917;
  --text-color: #1c1917;
  --muted-text-color: #57534e;
  --button-bg: #262626;
  --button-color: #fafafa;
  --button-hover-bg: #171717;
  --link-color: #2563eb;
  --link-hover-color: #1d4ed8;
  --overlay-color: rgba(10, 10, 10, 0.52);
  --shadow-color: rgba(0, 0, 0, 0.22);
  --ring-color: rgba(37, 99, 235, 0.28);
  --stake-status-healthy: #15803d;
  --stake-status-busy: #ca8a04;
  --stake-status-congested: #c2410c;
  --stake-status-degraded: #dc2626;
  --stake-status-incident: #b91c1c;

  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

html[data-theme="dark"] {
  color-scheme: dark;
  --background-color: #111111;
  --surface-color: #1a1a1a;
  --muted-surface-color: #252525;
  --border-color: #3f3f46;
  --primary-text-color: #f4f4f5;
  --text-color: #f4f4f5;
  --muted-text-color: #a1a1aa;
  --button-bg: #313131;
  --button-color: #f4f4f5;
  --button-hover-bg: #3c3c3c;
  --link-color: #93c5fd;
  --link-hover-color: #bfdbfe;
  --overlay-color: rgba(0, 0, 0, 0.68);
  --shadow-color: rgba(0, 0, 0, 0.45);
  --ring-color: rgba(147, 197, 253, 0.32);
  --stake-status-healthy: #34d399;
  --stake-status-busy: #fbbf24;
  --stake-status-congested: #fb923c;
  --stake-status-degraded: #f87171;
  --stake-status-incident: #ef4444;
}

/* Shared utility classes used by existing components */
.bg-neutral-primary {
  background-color: var(--surface-color);
}

.bg-neutral-secondary-soft,
.bg-neutral-secondary-medium {
  background-color: var(--muted-surface-color);
}

```

```
.text-heading,  
.text-primary {  
  color: var(--primary-text-color);  
}  
  
.text-body,  
.text-muted {  
  color: var(--muted-text-color);  
}  
  
.bg-surface,  
.bg-card {  
  background-color: var(--surface-color);  
}  
  
.border-divider {  
  border-color: var(--border-color);  
}  
  
.bg-tooltip {  
  background-color: var(--surface-color);  
  border: 1px solid var(--border-color);  
}  
  
.text-tooltip-text {  
  color: var(--text-color);  
}  
  
.text-success {  
  color: #16a34a;  
}  
  
.text-danger {  
  color: #dc2626;  
}  
  
.rounded-base {  
  border-radius: 0.5rem;  
}  
  
.focus\:ring-neutral-tertiary:focus,  
.focus\:ring-brand-medium:focus {  
  box-shadow: 0 0 0 3px var(--ring-color);  
}  
  
.div {  
  background-color: var(--background-color);  
}  
  
.select {  
  padding: 8px;  
  border-radius: 6px;  
  border: 1px solid var(--border-color);  
  background-color: var(--surface-color);  
  color: var(--text-color);  
  font-family: inherit;  
  font-size: 1em;  
}  
  
a {  
  font-weight: 500;  
  color: inherit;  
  text-decoration: none;  
}  
  
a:hover {  
  color: var(--link-hover-color);  
}  
  
body {
```

```
margin: 0;
min-width: 320px;
min-height: 100vh;
background-color: var(--background-color);
color: var(--text-color);
transition: background-color 0.25s ease, color 0.25s ease;
}

h1 {
  font-size: 3.2em;
  line-height: 1.1;
  color: var(--primary-text-color);
}

button {
  border-radius: 6px;
  border: 1px solid var(--border-color);
  padding: 0.6em 1.2em;
  font-size: 1em;
  font-weight: 500;
  font-family: inherit;
  background-color: var(--button-bg);
  color: var(--button-color);
  cursor: pointer;
  transition: all 0.2s ease;
}

button:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

/* Main button "base class" */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 0.35rem;
  white-space: nowrap;
  font-family: inherit;
  font-weight: 500;
  text-align: center;
  text-decoration: none;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: all 0.2s ease-in-out;
  margin: 4px 2px;
}

.btn-primary {
  background-color: #2563eb;
  color: #f9fafb;
}

.btn-primary:not(:disabled):hover {
  background-color: #1d4ed8;
}

.btn-secondary {
  background-color: var(--surface-color);
  color: var(--text-color);
  border: 1px solid var(--border-color);
}

.btn-secondary:not(:disabled):hover {
  background-color: var(--muted-surface-color);
}

.btn-green {
  background-color: #16a34a;
  color: #f9fafb;
}

.btn-green:not(:disabled):hover {
```

```
background-color: #15803d;
}

.btn-red {
background-color: #dc2626;
color: #f9fafb;
}
.btn-red:not(:disabled):hover {
background-color: #b91c1c;
}

.btn-sm {
padding: 0.5rem 1rem;
font-size: 0.875rem;
line-height: 1.25rem;
}

.btn-md {
padding: 0.75rem 1.25rem;
font-size: 1rem;
line-height: 1.5rem;
}

.btn-lg {
padding: 1rem 1.5rem;
font-size: 1.125rem;
line-height: 1.75rem;
}

.trade-confirm-overlay {
position: fixed;
inset: 0;
z-index: 45;
display: flex;
align-items: center;
justify-content: center;
padding: 1rem;
background: var(--overlay-color);
backdrop-filter: blur(2px);
}

.trade-confirm-panel {
width: 100%;
max-width: 28rem;
border-radius: 1rem;
border: 1px solid var(--border-color);
background: var(--surface-color);
color: var(--text-color);
padding: 1.5rem;
box-shadow: 0 24px 64px var(--shadow-color);
}

.trade-confirm-copy {
color: var(--muted-text-color);
}

.trade-confirm-summary {
border: 1px solid var(--border-color);
background: var(--muted-surface-color);
}

.stake-modal-overlay {
background: var(--overlay-color);
backdrop-filter: blur(6px);
}

.stake-modal-panel {
border-color: var(--border-color);
background: var(--surface-color);
color: var(--text-color);
box-shadow: 0 24px 64px var(--shadow-color);
}
```

```
}

.stake-modal-subpanel {
  border-color: var(--border-color);
  background: var(--background-color);
  color: var(--text-color);
}

.stake-status-healthy {
  color: var(--stake-status-healthy);
}

.stake-status-busy {
  color: var(--stake-status-busy);
}

.stake-status-congested {
  color: var(--stake-status-congested);
}

.stake-status-degraded {
  color: var(--stake-status-degraded);
}

.stake-status-incident {
  color: var(--stake-status-incident);
}

html[data-theme="dark"] .btn-green {
  background-color: #16a34a;
  color: #f9fafb;
}

html[data-theme="dark"] .btn-green:not(:disabled):hover {
  background-color: #15803d;
}

html[data-theme="dark"] .btn-red {
  background-color: #dc2626;
  color: #f9fafb;
}

html[data-theme="dark"] .btn-red:not(:disabled):hover {
  background-color: #b91c1c;
}

html[data-theme="dark"] button:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

html[data-theme="dark"] select {
  background-color: var(--surface-color);
  color: var(--text-color);
  border-color: var(--border-color);
}

html[data-theme="dark"] .trade-confirm-overlay {
  background: var(--overlay-color);
}

html[data-theme="dark"] .trade-confirm-panel {
  box-shadow: 0 24px 64px var(--shadow-color);
}
```

FILE: src/lib/chainStakeData.ts

```
export const CHAINS =
[
  {
    name: "Solana",
    staking: {
      baseApy: 7.5,
      volatility: 1.5,
      lockupDays: 90,
      unbondingDays: 2,
    },
    network: {
      avgHealth: 82,
      volatility: 12,
      incidentChance: 0.03,
    },
  },
  {
    name: "Polkadot",
    staking: {
      baseApy: 12,
      volatility: 2,
      lockupDays: 28,
      unbondingDays: 28,
    },
    network: {
      avgHealth: 90,
      volatility: 6,
      incidentChance: 0.01,
    },
  },
  {
    name: "Cardano",
    staking: {
      baseApy: 5,
      volatility: 1,
      lockupDays: 0,
      unbondingDays: 0,
    },
    network: {
      avgHealth: 88,
      volatility: 8,
      incidentChance: 0.02,
    },
  },
  {
    name: "Avalanche",
    staking: {
      baseApy: 9,
      volatility: 1.8,
      lockupDays: 14,
      unbondingDays: 14,
    },
    network: {
      avgHealth: 85,
      volatility: 10,
      incidentChance: 0.025,
    },
  },
  {
    name: "Ethereum",
    staking: {
      baseApy: 4.5,
      volatility: 1.2,
      lockupDays: 30,
      unbondingDays: 30,
    },
    network: {
      avgHealth: 92,
      volatility: 5,
      incidentChance: 0.008,
    },
  },
]
```

```
    },  
  },  
]
```

FILE: src/lib/healthToStatus.ts

```
import { type NetworkState } from "../types/network";
export function healthToStatus(score: number): NetworkState["status"] {
  if (score >= 90) return "healthy";
  if (score >= 75) return "busy";
  if (score >= 60) return "congested";
  if (score >= 40) return "degraded";
  return "incident";
}
```

FILE: src/lib/links.ts

```
export const Links = [
  {
    link: "/Dashboard",
    name: "Dashboard",
  },
  {
    link: "/Trade",
    name: "Trade",
  },
  {
    link: "/Stake",
    name: "Stake",
  },
  {
    link: "/Wallet-Dashboard",
    name: "Wallet dashboard",
  },
  {
    link: "/Leaderboard",
    name: "Leaderboard",
  },
  {
    link: "/Account",
    name: "Account"
  }
]
```

FILE: src/lib/supabaseClient.ts

```
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL as string | undefined;
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY as string | undefined;

if (!supabaseUrl || !supabaseAnonKey) {
  console.warn(
    'Supabase URL or anon key is missing. Set VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY in your .env file.'
  );
}

export const supabase = createClient(supabaseUrl ?? '', supabaseAnonKey ?? '');
```

FILE: src/main.tsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'
import { BrowserRouter } from 'react-router-dom'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

FILE: src/pages/Account.tsx

```

import { useEffect, useState } from 'react';
import { Check, X } from 'lucide-react';
import { useNavigate } from 'react-router-dom';
import Button from '../Components/Button';
import { useAuth } from '../hooks/useAuth';
import onboardCheck from '../utils/onboardcheck';
import fetchUserBadgeProgress from '../utils/userBadges';

type Mode = 'login' | 'signup';
type BadgePreview = {
  id: string;
  symbol: string;
  title: string;
  description: string;
  unlocked: boolean;
};
type BadgeSummary = {
  badges: BadgePreview[];
  xp: number;
  tradeCount: number;
};

const USERNAME_MIN = 6;
const USERNAME_MAX = 20;
const EMAIL_MIN = 8;
const EMAIL_MAX = 40;
const PASSWORD_MIN = 8;
const PASSWORD_MAX = 16;

const initialBadgeSummary: BadgeSummary = {
  badges: [],
  xp: 0,
  tradeCount: 0,
};

const Account = () => {
  const navigate = useNavigate();
  const { user, loading, signIn, signUp, signOut } = useAuth();

  const [mode, setMode] = useState<Mode>('login');
  const [username, setUsername] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState('');
  const [error, setError] = useState<string | null>(null);
  const [success, setSuccess] = useState<string | null>(null);
  const [submitting, setSubmitting] = useState(false);

  const [onboarded, setOnboarded] = useState(false);
  const [badgeSummary, setBadgeSummary] = useState<BadgeSummary>(initialBadgeSummary);
  const [accountDetailsLoading, setAccountDetailsLoading] = useState(false);

  let unlockedBadgeCount = 0;
  for (const badge of badgeSummary.badges) {
    if (badge.unlocked) {
      unlockedBadgeCount += 1;
    }
  }

  useEffect(() => {
    //Validate user
    if (!user) {
      setOnboarded(false);
      setBadgeSummary(initialBadgeSummary);
      setAccountDetailsLoading(false);
      return;
    }

    const loadAccountDetails = async () => {

```

```

    setAccountDetailsLoading(true);
    //Call onboardCheck and fetchUserBadgeProgress functions.
    const nextOnboarded = await onboardCheck(user);
    const nextBadgeSummary = await fetchUserBadgeProgress(user);
    //Set state variables
    setOnboarded(nextOnboarded);
    setBadgeSummary({
      badges: nextBadgeSummary.badges,
      xp: nextBadgeSummary.xp,
      tradeCount: nextBadgeSummary.tradeCount,
    });
    setAccountDetailsLoading(false);
  };
  //Check for any errors
  loadAccountDetails().catch((error) => {
    console.error('Error loading account details:', error);
    setAccountDetailsLoading(false);
  });
}, [user]);

const handleSubmit = async (event: React.FormEvent) => {
  event.preventDefault();

  setError(null);
  setSuccess(null);
  setSubmitting(true);
  // Trim each field (remove whitespaces)
  const trimmedUsername = username.trim();
  const trimmedEmail = email.trim();
  const trimmedPassword = password.trim();
  const trimmedConfirmPassword = confirmPassword.trim();
  //Check for then perform necessary frontend validation for signup and login
  if ((trimmedUsername.length < USERNAME_MIN || trimmedUsername.length > USERNAME_MAX) && mode ===
'signup') {
    setError(`Username must be ${USERNAME_MIN}-${USERNAME_MAX} characters.`);
    setSubmitting(false);
    return;
  }
  if (trimmedEmail.length < EMAIL_MIN || trimmedEmail.length > EMAIL_MAX) {
    setError(`Email must be ${EMAIL_MIN}-${EMAIL_MAX} characters.`);
    setSubmitting(false);
    return;
  }
  if (trimmedPassword.length < PASSWORD_MIN || trimmedPassword.length > PASSWORD_MAX) {
    setError(`Password must be ${PASSWORD_MIN}-${PASSWORD_MAX} characters.`);
    setSubmitting(false);
    return;
  }

  //If signup
  if (mode === 'signup'){
    if (trimmedPassword !== trimmedConfirmPassword) {
      setError('Passwords do not match.');
```

```

    }

    setSubmitting(false);
  };

const handleSignOut = async () => {
  setError(null);
  setSuccess(null);
  await signOut();
};

if (loading) {
  return (
    <div className="px-6 md:px-10 pb-10 flex items-center justify-center min-h-[60vh]">
      <div className="text-sm">Loading...</div>
    </div>
  );
}

if (user) {
  return (
    <div className="px-6 md:px-10 pb-10">
      <div className="max-w-5xl mx-auto space-y-6">
        <header className="space-y-2">
          <h1 className="text-3xl font-semibold">Account</h1>
          <p className="text-sm md:text-base text-[var(--muted-text-color)]">
            Manage your profile, onboarding status and achievement badges.
          </p>
        </header>

        <section className="grid grid-cols-1 md:grid-cols-2 gap-4">
          <div className="rounded-2xl border border-[var(--border-color)] bg-[var(--surface-color)] p-5 space-y-4">
            <h2 className="text-xl font-semibold">Profile</h2>
            <div className="space-y-2 text-sm">
              <p>
                <span className="font-medium">Username:</span>{' '}
                {user.user_metadata.username || user.email || 'User'}
              </p>
              <p>
                <span className="font-medium">Email:</span> {user.email || 'N/A'}
              </p>
            </div>
            <div className="grid grid-cols-3 gap-3 text-sm">
              <div className="rounded-lg border border-[var(--border-color)] p-3">
                <p className="text-xs text-[var(--muted-text-color)]">XP</p>
                <p className="text-lg font-semibold">{badgeSummary.xp.toLocaleString()}</p>
              </div>
              <div className="rounded-lg border border-[var(--border-color)] p-3">
                <p className="text-xs text-[var(--muted-text-color)]">Trades</p>
                <p className="text-lg font-semibold">{badgeSummary.tradeCount}</p>
              </div>
              <div className="rounded-lg border border-[var(--border-color)] p-3">
                <p className="text-xs text-[var(--muted-text-color)]">Badges</p>
                <p className="text-lg font-semibold">{unlockedBadgeCount}/5</p>
              </div>
            </div>
            <div>
              <Button onClick={handleSignOut} variant="primary" size="md" type="button">
                Log out
              </Button>
            </div>
          </div>

          <div className="rounded-2xl border border-[var(--border-color)] bg-[var(--surface-color)] p-5 space-y-4">
            <h2 className="text-xl font-semibold">Onboarding</h2>
            <p className="text-sm text-[var(--muted-text-color)]">
              Onboarding helps adapt chart and trading assistance to your experience level.
            </p>
            <p>{onboarded ? (

```

```

    <div className="flex items-center gap-2 font-medium text-green-600">
      <Check className="h-5 w-5" />
      <span>Onboarding complete</span>
    </div>
  ) : (
    <div className="space-y-3">
      <div className="flex items-center gap-2 font-medium text-red-600">
        <X className="h-5 w-5" />
        <span>Onboarding incomplete</span>
      </div>
      <Button
        type="button"
        variant="green"
        size="md"
        onClick={() => navigate('/onboarding')}
      >
        Start Onboarding
      </Button>
    </div>
  )}
</div>
</section>

<section className="rounded-2xl border border-[var(--border-color)] bg-[var(--surface-color)] p-5
space-y-4">
  <div className="flex items-center justify-between">
    <h2 className="text-xl font-semibold">Badges</h2>
    <p className="text-xs text-[var(--muted-text-color)]">{unlockedBadgeCount}/5 unlocked</p>
  </div>

  {accountDetailsLoading ? (
    <p className="text-sm text-[var(--muted-text-color)]">Loading badges...</p>
  ) : badgeSummary.badges.length === 0 ? (
    <p className="text-sm text-[var(--muted-text-color)]">No badges yet.</p>
  ) : (
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-3">
      {badgeSummary.badges.map((badge) => (
        <div
          key={badge.id}
          className={`rounded-md border p-3 ${
            badge.unlocked
              ? 'border-[var(--border-color)] bg-[var(--surface-color)]'
              : 'border-[var(--border-color)] bg-[var(--muted-surface-color)]'
          }`}
        >
          <div className="flex items-center gap-2">
            <span
              className={`inline-flex h-8 w-8 items-center justify-center rounded-md text-sm font-
semibold ${
                badge.unlocked
                  ? 'bg-[var(--muted-surface-color)] text-[var(--text-color)]'
                  : 'bg-[var(--surface-color)] text-[var(--muted-text-color)]'
              }`}
            >
              {badge.unlocked ? badge.symbol : '?'}
            </span>
            <p className="text-sm font-semibold">
              {badge.unlocked ? badge.title : 'Undiscovered Badge'}
            </p>
          </div>
          <p className="mt-2 text-xs text-[var(--muted-text-color)]">
            {badge.unlocked
              ? badge.description
              : 'Keep trading and earning XP to reveal this badge.'}
          </p>
        </div>
      )})}
    </div>
  )}
</div>
</section>

```

```

    </div>
  </div>
);
}

return (
  <div className="px-6 md:px-10 pb-10 flex items-center justify-center min-h-[60vh]">
    <div className="max-w-2xl w-full rounded-2xl border border-[var(--border-color)] bg-[var(--surface-color)] p-6 md:p-8">
      <h1 className="text-3xl font-semibold mb-4">
        {mode === 'login' ? 'Welcome back' : 'Create your account'}
      </h1>
      <p className="text-sm md:text-base text-[var(--muted-text-color)] mb-6">
        {mode === 'login'
          ? 'Use email and password to access trading, portfolio and leaderboard features.'
          : 'Sign up with email and password to start trading, building your portfolio and climbing the
leaderboard.'}
      </p>

      <div className="flex mb-4 w-full">
        <Button
          type="button"
          onClick={() => setMode('login')}
          variant={` ${mode === 'login' ? 'secondary' : 'primary'} `}
          size="md"
          className="w-1/2"
        >
          Log in
        </Button>
        <Button
          type="button"
          onClick={() => setMode('signup')}
          variant={` ${mode === 'login' ? 'primary' : 'secondary'} `}
          size="md"
          className="w-1/2"
        >
          Sign up
        </Button>
      </div>

      <form onSubmit={handleSubmit} className="space-y-4">
        {mode === 'signup' && (
          <div className="flex flex-col gap-1">
            <label className="text-sm font-medium">Username</label>
            <input
              type="text"
              className="border rounded-md px-3 py-2"
              placeholder="6-20 characters"
              value={username}
              onChange={(event) => setUsername(event.target.value)}
              minLength={USERNAME_MIN}
              maxLength={USERNAME_MAX}
              required
            />
          </div>
        )}

        <div className="flex flex-col gap-1">
          <label className="text-sm font-medium">Email</label>
          <input
            type="email"
            value={email}
            onChange={(event) => setEmail(event.target.value)}
            className="border rounded-md px-3 py-2"
            placeholder="you@example.com"
            minLength={mode === 'signup' ? EMAIL_MIN : undefined}
            maxLength={mode === 'signup' ? EMAIL_MAX : undefined}
            required
          />
        </div>
      </form>
    </div>
  </div>
);

```

```

<div className="flex flex-col gap-1">
  <label className="text-sm font-medium">Password</label>
  <input
    type="password"
    value={password}
    onChange={(event) => setPassword(event.target.value)}
    className="border rounded-md px-3 py-2"
    placeholder={mode === 'signup' ? '8-16 characters' : 'Your password'}
    required
    minLength={mode === 'signup' ? PASSWORD_MIN : 1}
    maxLength={mode === 'signup' ? PASSWORD_MAX : undefined}
  />
</div>

{mode === 'signup' && (
  <div className="flex flex-col gap-1">
    <label className="text-sm font-medium">Confirm Password</label>
    <input
      type="password"
      value={confirmPassword}
      onChange={(event) => setConfirmPassword(event.target.value)}
      className="border rounded-md px-3 py-2"
      placeholder="Re-enter (8-16 chars)"
      required
      minLength={PASSWORD_MIN}
      maxLength={PASSWORD_MAX}
    />
  </div>
)}

{error && <p className="text-sm text-red-600">{error}</p>}
{success && (
  <p className="text-sm text-green-600">{success}</p>
)}

<Button
  type="submit"
  disabled={submitting}
  variant="primary"
  size="md"
>
  {submitting
    ? mode === 'login'
      ? 'Logging in...'
      : 'Creating account...'
    : mode === 'login'
      ? 'Log in'
      : 'Sign up'}
</Button>
</form>
</div>
</div>
);
};

export default Account;

```

FILE: src/pages/Dashboard.tsx

```

import CryptoChart from '../Components/CryptoChart';

const Dashboard = () => {
  return (
    <div className="px-6 md:px-10 pb-10">
      <div className="max-w-6xl mx-auto space-y-6">
        <header className="flex flex-col gap-2">
          <h1 className="text-3xl font-semibold">
            Dashboard
          </h1>
          <p className="text-sm md:text-base text-[var(--muted-text-color)] max-w-2xl">
            Monitor the market and keep an eye on key pairs before you place a simulated
            order.
          </p>
        </header>

        <section className="grid grid-cols-1 md:grid-cols-3 gap-4 text-sm">
          <div className="rounded-2xl shadow-sm border border-[var(--border-color)] bg-[var(--surface-color)]
p-4">
            <p className="text-xs text-[var(--muted-text-color)] mb-1">Total practice balance</p>
            <p className="text-2xl font-semibold">
              100,000 <span className="text-xs text-[var(--muted-text-color)] ml-1">USDT</span>
            </p>
            <p className="text-xs text-[var(--muted-text-color)] mt-2">
              Starting balance used across your simulated account.
            </p>
          </div>
          <div className="rounded-2xl shadow-sm border border-[var(--border-color)] bg-[var(--surface-color)]
p-4">
            <p className="text-xs text-[var(--muted-text-color)] mb-1">Open simulated positions</p>
            <p className="text-2xl font-semibold">
              0
            </p>
            <p className="text-xs text-[var(--muted-text-color)] mt-2">
              Once you place trades, you could surface them here.
            </p>
          </div>
          <div className="rounded-2xl shadow-sm border border-[var(--border-color)] bg-[var(--surface-color)]
p-4">
            <p className="text-xs text-[var(--muted-text-color)] mb-1">Focus pairs</p>
            <p className="text-2xl font-semibold">
              BTC / ETH / SOL
            </p>
            <p className="text-xs text-[var(--muted-text-color)] mt-2">
              Core pairs for demonstrating volatility and risk.
            </p>
          </div>
        </section>

        <section className="rounded-2xl shadow-sm border border-[var(--border-color)] bg-[var(--surface-color)]
p-4 md:p-6">
          <p className="text-sm font-medium text-[var(--muted-text-color)] mb-4">
            Market overview
          </p>
          <CryptoChart />
        </section>
      </div>
    </div>
  );
};

export default Dashboard;

```

FILE: src/pages/Home.tsx

```

import { useNavigate } from 'react-router-dom';

const Home = () => {
  const navigate = useNavigate();

  return (
    <div className="w-full min-h-[80vh] flex items-center justify-center px-6 pb-16">
      <div className="max-w-6xl w-full grid grid-cols-1 lg:grid-cols-2 gap-10 items-center">
        <div>
          <p className="text-sm font-semibold text-gray-600 uppercase mb-2">
            Educational crypto simulator
          </p>
          <h1 className="text-4xl md:text-5xl font-semibold mb-4 ">
            Learn to trade crypto without risking real money.
          </h1>
          <p className="text-base md:text-lg mb-6 max-w-xl">
            Block Finance lets you practise placing orders, managing a portfolio, and
            exploring the market using realistic data and a clean, usable interface.
          </p>
          <div className="flex flex-wrap gap-3 mb-8">
            <button
              type="button"
              className="btn btn-lg btn-primary"
              onClick={() => navigate('/Trade')}
            >
              Start trading
            </button>
            <button
              type="button"
              className="btn btn-lg btn-secondary"
              onClick={() => navigate('/Account')}
            >
              Log in / Sign up
            </button>
          </div>
          <div className="grid grid-cols-3 gap-4 text-sm ">
            <div>
              <p className="font-semibold ">Simulated</p>
              <p>No real funds or live orders are used.</p>
            </div>
            <div>
              <p className="font-semibold ">Guided</p>
              <p>Focus on concepts: orders, risk and P&L.</p>
            </div>
            <div>
              <p className="font-semibold ">Beginner & Advanced Friendly</p>
              <p>Great for all skill-levels</p>
            </div>
          </div>
        </div>
        <div className="rounded-2xl border border-[var(--border-color)] bg-[var(--surface-color)] backdrop-blur
shadow-md p-6 md:p-8">
          <p className="text-sm font-medium mb-4">
            Quick overview
          </p>
          <div className="grid grid-cols-2 gap-4 mb-6 text-sm">
            <div className="rounded-xl border border-[var(--border-color)] p-4">
              <p className="text-xs text-[var(--muted-text-color)] mb-1">Practice balance</p>
              <p className="text-2xl font-semibold ">
                100,000<span className="text-xs text-[var(--muted-text-color)] ml-1">USDT</span>
              </p>
            </div>
            <div className="rounded-xl border border-[var(--border-color)] p-4">
              <p className="text-xs text-[var(--muted-text-color)] mb-1">Pairs available</p>
              <p className="text-2xl font-semibold ">
                6+
              </p>
            </div>
          </div>
        </div>
      </div>
    </div>
  )
}

```

```

    </p>
  </div>
  <div className="rounded-xl border border-[var(--border-color)] bg-[var(--muted-surface-color)] p-
4">
    <p className="text-xs ">
      Risk-free learning
    </p>
    <p className="text-sm ">
      Place buys & sells and watch how your positions change.
    </p>
  </div>
  <div className="rounded-xl border border-[var(--border-color)] bg-[var(--muted-surface-color)] p-
4">
    <p className="text-xs ">
      Real-time data
    </p>
    <p className="text-sm ">
      Experience live market movements and trends.
    </p>
  </div>
  <div>
    <p className="text-xs text-[var(--muted-text-color)]">
      To get started, create an account, choose your skill level and then place a simulated trade on
      BTC/USDT or
      another pair of your choice.
    </p>
  </div>
</div>
</div>
);
};

export default Home;

```

FILE: src/pages/Leaderboard.tsx

```

import { useEffect, useState } from 'react';
import Button from '../Components/Button';
import { useAuth } from '../hooks/useAuth';
import { fetchLeaderboardData, {
  buildLeaderboardRows,
  type LeaderboardRow,
  type LeaderboardUser,
} } from '../utils/leaderboardUtils';

// Formats portfolio numbers as currency for display.
const formatCurrency = (value: number) =>
  value.toLocaleString(undefined, {
    minimumFractionDigits: 2,
    maximumFractionDigits: 2,
  });

// Returns the current table/card label based on metric + timeframe.
type Metric = 'xp' | 'PnL';
type Timeframe = '7d' | 'all-time';

const scoreLabel = (metric: Metric, timeframe: Timeframe) => {
  if (metric === 'xp') {
    return timeframe === '7d' ? 'XP (7D)' : 'XP (All-time)';
  }
  return timeframe === '7d' ? 'PnL (7D)' : 'PnL (All-time)';
};

// Picks the wallet name field that matches the selected timeframe.
const portfolioWalletName = (row: LeaderboardRow, timeframe: Timeframe) =>
  timeframe === '7d' ? row.portfolio7dWalletName : row.portfolioAllTimeWalletName;

const Leaderboard = () => {
  const { user } = useAuth();

  // UI filters and fetch state used across the page.
  const [metric, setMetric] = useState<Metric>('PnL');
  const [timeframe, setTimeframe] = useState<Timeframe>('all-time');
  const [users, setUsers] = useState<LeaderboardUser[]>([]);
  const [loading, setLoading] = useState(true);
  const [errorMessage, setErrorMessage] = useState('');

  // Load all leaderboard source data once when this page mounts.
  // Filtering/sorting for metric/timeframe is done client-side from this data.
  useEffect(() => {
    setLoading(true);
    setErrorMessage('');

    fetchLeaderboardData()
      .then((nextUsers) => {
        setUsers(nextUsers);
      })
      .catch((error) => {
        const fallback = error instanceof Error ? error.message : 'Unable to load leaderboard right now.';
        setErrorMessage(fallback);
      })
      .finally(() => {
        setLoading(false);
      });
  }, []);

  // Build ranked rows for current filter selection.
  const rows = buildLeaderboardRows(users, metric, timeframe);
  // Current signed-in user's row (for "Your Position" card).
  const myRow = user ? rows.find((row) => row.userId === user.id) ?? null : null;
  // Show top 10 in the main table.
  const topRows = rows.slice(0, 10);
  // Rank #1 row shown in "Top Performer" card.
  const topPerformer = rows[0] ?? null;

```

```

return (
  <div className="px-6 md:px-10 pb-10">
    <div className="max-w-5xl mx-auto space-y-6">
      <header className="space-y-2">
        <h1 className="text-3xl font-semibold">Leaderboard</h1>
        <p className="text-sm md:text-base max-w-2xl">
          Compare users by XP and portfolio performance across 7D and all-time rankings.
        </p>
      </header>

      { /* Filter panel: metric (XP/Portfolio) and timeframe (7D/All-time). */ }
      <section className="rounded-2xl border border-[var(--border-color)] bg-[var(--surface-color)] p-4 md:p-
5 space-y-4">
        <div className="flex flex-col gap-4 md:flex-row md:items-end md:justify-between">
          <div className="space-y-2">
            <p className="text-xs uppercase ">Main Filter</p>
            <div className="flex flex-wrap gap-2">
              <Button
                type="button"
                size="sm"
                variant={metric === 'xp' ? 'primary' : 'secondary'}
                className="!m-0"
                onClick={() => setMetric('xp')}
              >
                XP
              </Button>
              <Button
                type="button"
                size="sm"
                variant={metric === 'PnL' ? 'primary' : 'secondary'}
                className="!m-0"
                onClick={() => setMetric('PnL')}
              >
                Portfolio
              </Button>
            </div>
          </div>

          <div className="space-y-2">
            <p className="text-xs uppercase ">Timeframe</p>
            <div className="flex flex-wrap gap-2">
              <Button
                type="button"
                size="sm"
                variant={timeframe === '7d' ? 'primary' : 'secondary'}
                className="!m-0"
                onClick={() => setTimeframe('7d')}
              >
                7D
              </Button>
              <Button
                type="button"
                size="sm"
                variant={timeframe === 'all-time' ? 'primary' : 'secondary'}
                className="!m-0"
                onClick={() => setTimeframe('all-time')}
              >
                All-time
              </Button>
            </div>
          </div>
        </div>
      </section>

      { /* Summary cards: signed-in user position and current top performer. */ }
      <section className="grid gap-4 md:grid-cols-2">
        <div className="rounded-2xl border border-[var(--border-color)] bg-[var(--surface-color)] p-4">
          <p className="text-xs uppercase ">Your Position</p>
          {myRow ? (

```

```

<>
<p className="mt-2 text-2xl font-semibold">#{myRow.rank}</p>
<p className="mt-1 text-sm">{scoreLabel(metric, timeframe)}</p>
<p className="text-lg font-medium mt-1">
  {metric === 'xp' ? (
    <span>{Math.round(myRow.score).toLocaleString()}</span>
  ) : (
    <span
      className={`inline-flex items-center gap-1 ${
        myRow.score >= 0
          ? 'text-green-600'
          : 'text-red-600'
      }`}
    >
      <span>{myRow.score >= 0 ? '↑' : '↓'}</span>
      <span>{`$${myRow.score >= 0 ? '+' : '-'}${formatCurrency(Math.abs(myRow.score))}`}</span>
    </span>
  )}
</p>
{metric === 'PnL' && portfolioWalletName(myRow, timeframe) !== '' && (
  <p className="text-xs mt-1">
    Best wallet: {portfolioWalletName(myRow, timeframe)}
  </p>
)}
</>
) : (
  <p className="mt-2 text-sm">Sign in and start trading to appear in the leaderboard.</p>
)
</div>

<div className="rounded-2xl border border-[var(--border-color)] bg-[var(--surface-color)] p-4">
  <p className="text-xs uppercase">Top Performer</p>
  {topPerformer ? (
    <>
      <p className="mt-2 text-xl font-semibold">{topPerformer.username}</p>
      <p className="mt-1 text-sm">{scoreLabel(metric, timeframe)}</p>
      <p className="text-lg font-medium mt-1">
        {metric === 'xp' ? (
          <span>{Math.round(topPerformer.score).toLocaleString()}</span>
        ) : (
          <span
            className={`inline-flex items-center gap-1 ${
              topPerformer.score >= 0
                ? 'text-green-600'
                : 'text-red-600'
            }`}
          >
            <span>{topPerformer.score >= 0 ? '↑' : '↓'}</span>
            <span>{`$${topPerformer.score >= 0 ? '+' : '-'}
'`${formatCurrency(Math.abs(topPerformer.score))}`}</span>
          </span>
        )}
      </p>
      {metric === 'PnL' && portfolioWalletName(topPerformer, timeframe) !== '' && (
        <p className="text-xs mt-1">
          Wallet: {portfolioWalletName(topPerformer, timeframe)}
        </p>
      )}
    </>
  ) : (
    <p className="mt-2 text-sm">No users found yet.</p>
  )}
</div>
</section>

{ /* Main ranking table with loading, error and empty states. */ }
<section className="rounded-2xl border border-[var(--border-color)] bg-[var(--surface-color)] p-4 md:p-
5">
  <div className="flex items-center justify-between mb-3">
    <h2 className="text-lg font-semibold">Top Performers</h2>
    <p className="text-xs">{scoreLabel(metric, timeframe)}</p>
  </div>

```

```

</div>

{/* While data is loading, show a simple status message. */}
{loading ? (
  <p className="text-sm">Loading leaderboard...</p>
  /* If fetch failed, show server/client error text. */
) : errorMessage !== '' ? (
  <p className="text-sm text-red-600">{errorMessage}</p>
  /* No rows after fetch. */
) : topRows.length === 0 ? (
  <p className="text-sm">No leaderboard data yet.</p>
) : (
  // Success state: render ranked table rows.
  <div className="overflow-x-auto">
    <table className="min-w-full text-sm">
      <thead>
        <tr className="text-left text-xs uppercase">
          <th className="py-2 pr-3">Rank</th>
          <th className="py-2 pr-3">User</th>
          <th className="py-2 text-right">{scoreLabel(metric, timeframe)}</th>
        </tr>
      </thead>
      <tbody>
        {topRows.map((row) => {
          // Highlight the signed-in user's row for quick visibility.
          const isCurrentUser = user?.id === row.userId;
          return (
            <tr
              key={row.userId}
              className={border-t border-[var(--border-color)] ${
                isCurrentUser ? 'bg-gray-200/60' : ''
              }}
            >
              <td className="py-3 pr-3 font-medium">#{row.rank}</td>
              <td className="py-3 pr-3">
                <p className={`font-medium ${isCurrentUser ? 'text-gray-800' : ''}`}>
                  {row.username}
                  {isCurrentUser ? ' (You)' : ''}
                </p>
                {metric === 'PnL' && (
                  <p className="text-xs">
                    Wallet: {portfolioWalletName(row, timeframe) ? portfolioWalletName(row, timeframe)
: "No wallet created"}
                  </p>
                )}
                {row.email !== '' && <p className="text-xs">{row.email}</p>}
              </td>
              <td className="py-3 text-right font-medium">
                {metric === 'xp' ? (
                  <span>{Math.round(row.score).toLocaleString()}</span>
                ) : (
                  <span
                    className={`inline-flex items-center gap-1 ${
                      row.score >= 0
                        ? 'text-green-600'
                        : 'text-red-600'
                    }`}
                  >
                    <span>{row.score >= 0 ? '↑' : '↓'}</span>
                    <span>{`${row.score >= 0 ? '+' : '-'}${formatCurrency(Math.abs(row.score))}`}</span>
                  </span>
                )}
              </td>
            </tr>
          );
        })}
      </tbody>
    </table>
  </div>
)}

```

```
        </section>
      </div>
    </div>
  );
};

export default Leaderboard;
```

FILE: src/pages/Onboarding.tsx

```

import React, { useEffect, useState } from 'react'
import { AlertTriangle, Shield, TrendingUp, Brain } from 'lucide-react'
import Button from '../Components/Button'
import Tooltip from '../Components/Tooltip'
import { useNavigate } from 'react-router-dom'
import { useAuth } from '../hooks/useAuth'
import updateSkillLevel from '../utils/UpdateSkillLevel'
import onboardCheck from '../utils/onboardcheck'

const Onboarding = () => {
  const navigate = useNavigate();
  const { user } = useAuth();
  const [onboarded, setOnboarded] = useState(false);
  useEffect(() => {
    // Check if user exists before calling onboardCheck
    if (!user) return;
    // Call the onboardCheck function and update the onboarded state
    onboardCheck(user).then(setOnboarded);
  }, [user]);
  if (onboarded) {
    navigate('/Dashboard');
  }
  //User initially starts at the first section, then increments as they proceed
  const [currentSection, setCurrentSection] = useState(0)
  // User's selected skill level at the end of the form (passed into database after onboarding is complete)
  const [skillLevel, setSkillLevel] = useState('')
  // Track which acknowledgments the user has checked
  const [acknowledged, setAcknowledged] = useState({
    risks: false,
    scams: false
  })

  const sections = [
    {
      title: "Understanding Financial Risks",
      icon: <TrendingUp className="w-12 h-12 text-orange-500" />,
      content: (
        <div className="space-y-4">
          <h3 className="text-xl font-semibold">Security & Financial Risks</h3>
          <div className="space-y-3">
            <div className="bg-orange-50 p-4 rounded-lg border border-orange-200">

              <h4 className="font-semibold mb-2 flex items-center gap-2">
                <AlertTriangle className="w-5 h-5 text-orange-600" />
                Volatility
              </h4>
              <p className="text-sm">Cryptocurrency prices can fluctuate dramatically within hours. You may lose a significant portion of your investment quickly.</p>
            </div>

            <div className="bg-orange-50 p-4 rounded-lg border border-orange-200">
              <h4 className="font-semibold">
                <Tooltip text = "Staking lock-up periods are when you agree to keep your digital money in a special online pot for a set amount of time." position = "top">
                  Staking Lock-Up Periods
                </Tooltip>
              </h4>

              <p className="text-sm mt-3">When staking, your funds may be locked for days, weeks, or months. You cannot access or sell them during this period, even if prices drop.</p>
            </div>

            <div className="bg-orange-50 p-4 rounded-lg border border-orange-200">
              <h4 className="font-semibold mb-2 flex items-center gap-2">
                <AlertTriangle className="w-5 h-5 text-orange-600" />
                Loss of Access
            
```

```

    </h4>
    <p className="text-sm">
      If you lose your <Tooltip text="Private keys are secret codes that prove you own your
      cryptocurrency. They're like a password that can never be reset." position = "bottom">private keys</Tooltip> or
      <Tooltip text="A seed phrase is a list of 12-24 words that can restore access to your wallet. Write it down and
      store it safely offline.">seed phrase</Tooltip>, your crypto is gone forever. There's no customer service to
      recover it.
    </p>
  </div>

  <div className="bg-orange-50 p-4 rounded-lg border border-orange-200">
    <h4 className="font-semibold mb-2">Regulatory Risk</h4>
    <p className="text-sm">Cryptocurrency regulations are evolving. Changes in laws could impact your
    holdings or ability to trade.</p>
  </div>
</div>

<label className="flex items-center gap-2 mt-6 cursor-pointer">
  <input
    type="checkbox"
    checked={acknowledged.risks}
    onChange={(e) => setAcknowledged({...acknowledged, risks: e.target.checked})}
    className="w-4 h-4"
  />
  <span className="text-sm font-medium ">I understand these financial risks</span>
</label>
</div>
)
},
{
  title: "Avoiding Crypto Scams",
  icon: <Shield className="w-12 h-12 text-red-500" />,
  content: (
    <div className="space-y-4">
      <h3 className="text-xl font-semibold ">How Scams Work & How to Stay Safe</h3>
      <div className="space-y-3">
        <div className="bg-red-50 p-4 rounded-lg border border-red-200">
          <h4 className="font-semibold mb-2 text-red-800">Common Scam Types</h4>
          <ul className="text-sm space-y-1">
            <li>Fake tokens promising guaranteed returns</li>
            <li>Phishing websites that steal your credentials</li>
            <li>Impersonators pretending to be support staff</li>
            <li>Pump and dump schemes on unknown coins</li>
            <li>"Giveaway" scams asking you to send crypto first</li>
          </ul>
        </div>

        <div className="bg-green-50 p-4 rounded-lg border border-green-200">
          <h4 className="font-semibold mb-2 text-green-800">How to Protect Yourself</h4>
          <ul className="text-sm space-y-2">
            <li className="flex items-start gap-2">
              <span className="text-green-600 font-bold"></span>
              <span><strong>Stick to established coins</strong> - Avoid brand new tokens with no track
              record</span>
            </li>
            <li className="flex items-start gap-2">
              <span className="text-green-600 font-bold"></span>
              <span><strong>If it seems too good to be true, it is</strong> - No legitimate investment
              guarantees 10x returns</span>
            </li>
            <li className="flex items-start gap-2">
              <span className="text-green-600 font-bold"></span>
              <span><strong>Never share your private keys</strong> - No legitimate service will ever ask for
              them</span>
            </li>
            <li className="flex items-start gap-2">
              <span className="text-green-600 font-bold"></span>
              <span><strong>Double-check URLs</strong> - Scammers create fake websites that look real</span>
            </li>
            <li className="flex items-start gap-2">
              <span className="text-green-600 font-bold"></span>

```

```

        <span><strong>Research before investing</strong> - Read reviews, check communities, verify
projects</span>
      </li>
    </ul>
  </div>
</div>

<label className="flex items-center gap-2 mt-6 cursor-pointer">
  <input
    type="checkbox"
    checked={acknowledged.scams}
    onChange={(e) => setAcknowledged({...acknowledged, scams: e.target.checked})}
    className="w-4 h-4"
  />
  <span className="text-sm font-medium ">I understand how to avoid scams</span>
</label>
</div>
)
},
{
  title: "Your Experience Level",
  icon: <Brain className="w-12 h-12 text-blue-500" />,
  content: (
    <div className="space-y-4">
      <h3 className="text-xl font-semibold ">Tell us about your crypto experience</h3>
      <p className="text-sm">This helps us tailor your experience and provide appropriate guidance.</p>

      <div className="space-y-3 mt-6">
        <label className="flex items-start gap-3 p-4 border-2 rounded-lg cursor-pointer transition-colors">
          <input
            type="radio"
            name="skillLevel"
            value="beginner"
            checked={skillLevel === 'beginner'}
            onChange={(e) => setSkillLevel(e.target.value)}
            className="mt-1"
          />
          <div>
            <div className="font-semibold ">Beginner</div>
            <div className="text-sm ">I'm new to cryptocurrency and blockchain technology</div>
          </div>
        </label>

        <label className="flex items-start gap-3 p-4 border-2 rounded-lg cursor-pointer transition-colors">
          <input
            type="radio"
            name="skillLevel"
            value="intermediate"
            checked={skillLevel === 'intermediate'}
            onChange={(e) => setSkillLevel(e.target.value)}
            className="mt-1"
          />
          <div>
            <div className="font-semibold ">Intermediate</div>
            <div className="text-sm ">I've traded or held crypto before and understand the basics</div>
          </div>
        </label>

      </div>
    </div>
  )
}
]

const canProceed = () => {
  if (currentSection === 0) return acknowledged.risks
  if (currentSection === 1) return acknowledged.scams
  if (currentSection === 2) return skillLevel !== ''
  return true
}

```

```

    }

    const handleNext = () => {
      if (currentSection < sections.length - 1) {
        setCurrentSection(currentSection + 1)
      } else {
        // Onboarding complete (will insert function to handle updating the database)
        updateSkillLevel(user, skillLevel).then((success) => {
          if (success) {
            // Redirect to dashboard after successful onboarding
            navigate('/Dashboard')
          } else {
            // Handle error (e.g., show a notification)
            console.log('Error updating onboarding status')
          }
        })
      }
    }

    return (
      <div className="min-h-screen py-12 px-4">
        <div className="max-w-3xl mx-auto">
          <div className="rounded-2xl border border-[var(--border-color)] bg-[var(--surface-color)] p-8">
            <div className="flex items-center justify-between mb-8">
              {sections.map((_, index) => (
                <React.Fragment key={index}>
                  <div className={`flex items-center justify-center w-10 h-10 rounded-full font-semibold ${
                    // If index is less than or equal to currentSection, apply completed styles
                    index <= currentSection
                      ? 'bg-blue-500 text-white'
                      : 'bg-[var(--muted-surface-color)] text-[var(--muted-text-color)]'
                  }`}>
                    {index + 1}
                  </div>
                  {/* Two bars that show progress, index < sections.length - 1 so that it doesn't show another bar
for the last item*/}
                  {index < sections.length - 1 && (
                    <div className={`flex-1 h-1 mx-3 ${
                      index < currentSection ? 'bg-[var(--button-bg)]' : 'bg-[var(--muted-surface-color)]'
                    }`} />
                    )}
                </React.Fragment>
              )})}
            </div>
            <div className="flex flex-col items-center mb-8">
              {sections[currentSection].icon}
              <h2 className="text-2xl font-bold mt-4">
                {sections[currentSection].title}
              </h2>
            </div>
            <div className="mt-10">
              {sections[currentSection].content}
            </div>
            {/* Section buttons*/}
            <div className="flex justify-between items-center pt-6 ">
              <Button
                type = "button"
                size = "md"
                onClick={() => setCurrentSection( currentSection - 1)}
                disabled={currentSection === 0}
                variant = "primary">
                Back
              </Button>

              <Button
                type = "button"
                size = "md"
                variant = "secondary"
                onClick={handleNext}
                disabled={!canProceed()}
              >

```

```
        {currentSection === sections.length - 1 ? 'Complete' : 'Next'}
      </Button>
    </div>
  </div>
</div>
</div>
)
}

export default Onboarding
```

FILE: src/pages/Stake.tsx

```

import { useEffect, useState } from 'react';
import { AlertTriangle } from 'lucide-react';
import Button from '../Components/Button';
import Tooltip from '../Components/Tooltip';
import { useAuth } from '../hooks/useAuth';
import { useTheme } from '../hooks/useTheme';
import stakeProvidersByChain from '../data/stakeProviders';
import { coinNameToTicker } from '../data/tickers';
import { CHAINS } from '../lib/chainStakeData';
import { healthToStatus } from '../lib/healthToStatus';
import type { StakeMessage } from '../types/stakeUi';
import type { wallet } from '../types/Wallet';
import placeStake from '../utils/actions/placeStake';
import fetchWalletStakes, { type WalletStake } from '../utils/FetchWalletStakes';
import { fetchStakeWalletData } from '../utils/stakePageUtils';
import sellStake from '../utils/actions/sellStake';

const formatCurrency = (value: number) => Number(value || 0).toLocaleString();

const inputClass =
  'w-full rounded-xl border border-[var(--border-color)] bg-[var(--background-color)] px-3 py-2 text-[var(--text-color)] focus:outline-none focus:ring-2 focus:ring-gray-400/30';

const healthMap = {
  healthy: { label: 'Online', className: 'stake-status-healthy' },
  busy: { label: 'Busy', className: 'stake-status-busy' },
  congested: { label: 'Congested', className: 'stake-status-congested' },
  degraded: { label: 'Degraded', className: 'stake-status-degraded' },
  incident: { label: 'Incident', className: 'stake-status-incident' },
} as const;

const getTicker = (chainName: string) =>
  coinNameToTicker[chainName.toLowerCase()] ?? chainName.slice(0, 3).toUpperCase();

const getLockDays = (ticker: string) => {
  const normalizedTicker = ticker.toUpperCase();
  for (const chain of CHAINS) {
    if (getTicker(chain.name) === normalizedTicker) {
      return chain.staking.lockupDays;
    }
  }

  return 0;
};

const getDaysLeft = (createdAt: string, lockupDays: number) => {
  const createdTime = new Date(createdAt).getTime();
  const elapsedDays = Math.floor((Date.now() - createdTime) / (1000 * 60 * 60 * 24));
  const remaining = lockupDays - elapsedDays;
  return remaining > 0 ? remaining : 0;
};

const Stake = () => {
  const { user } = useAuth();
  const { isDark } = useTheme();
  const [chainName, setChainName] = useState<string | null>(null);
  const [validatorId, setValidatorId] = useState('');
  const [walletId, setWalletId] = useState('');
  const [walletList, setWalletList] = useState<wallet[]>([]);
  const [stakeList, setStakeList] = useState<WalletStake[]>([]);
  const [isLoading, setIsLoading] = useState(false);
  const [amountText, setAmountText] = useState('10000');
  const [percentText, setPercentText] = useState('');
  const [isModalOpen, setIsModalOpen] = useState(false);
  const [isSavingStake, setIsSavingStake] = useState(false);
  const [sellingId, setSellingId] = useState<number | null>(null);
  const [statusMessage, setStatusMessage] = useState<StakeMessage>({ type: '', message: '' });
  const [sellStakeItem, setSellStakeItem] = useState<WalletStake | null>(null);
  const [sellDaysLeft, setSellDaysLeft] = useState(0);

```

```

const logoApiKey = import.meta.env.VITE_LOGO_API;

const chain = CHAINS.find((chainItem) => chainItem.name === chainName) ?? null;
const validatorOptions = chain ? stakeProvidersByChain[chain.name] ?? [] : [];
const validator = validatorOptions.find((validatorItem) => validatorItem.id === validatorId) ??
validatorOptions[0] ?? null;
const wallet = walletList.find((walletItem) => String(walletItem.walletid) === walletId) ?? null;
const amountValue = Number(amountText);
const stakeUsd = Number.isFinite(amountValue) ? amountValue : 0;
const walletUsd = Number(wallet?.usdt_balance ?? 0);

const onAmountChange = (value: string) => {
  setAmountText(value);

  const amount = Number(value);
  if (!Number.isFinite(amount) || amount <= 0 || walletUsd <= 0) {
    setPercentText('');
    return;
  }

  const percent = Math.min(100, Math.max(0, (amount / walletUsd) * 100));
  setPercentText(String(Math.round(percent)));
};

const onPercentChange = (value: string) => {
  setPercentText(value);

  const percent = Number(value);
  if (!Number.isFinite(percent) || walletUsd <= 0 || percent < 0) {
    setAmountText('');
    return;
  }

  const clampedPercent = Math.min(100, Math.max(0, percent));
  const amount = walletUsd * (clampedPercent / 100);
  setPercentText(String(clampedPercent));
  setAmountText(String(amount));
};

const setPercent = (percent: number) => {
  const clamped = Math.min(100, Math.max(0, percent));
  setPercentText(String(clamped));
  setAmountText(String((walletUsd * clamped) / 100));
};

const loadPage = async (preferredWalletId: string) => {
  if (!user) {
    setWalletList([]);
    setWalletId('');
    setStakeList([]);
    return;
  }

  setIsLoading(true);
  try {
    const { nextWallets, selectedWalletId } = await fetchStakeWalletData(user, preferredWalletId);
    setWalletList(nextWallets);
    setWalletId(selectedWalletId);

    const walletIds: Array<string | number> = [];
    for (const walletItem of nextWallets) {
      walletIds.push(walletItem.walletid);
    }
    const nextStakes = await fetchWalletStakes(walletIds, 'active');
    setStakeList(nextStakes);
  } finally {
    setIsLoading(false);
  }
};

```

```

useEffect(() => {
  if (!user) {
    setWalletList([]);
    setWalletId('');
    setStakeList([]);
    return;
  }

  const loadInitialData = async () => {
    setIsLoading(true);
    try {
      const { nextWallets, selectedWalletId } = await fetchStakeWalletData(user, '');
      setWalletList(nextWallets);
      setWalletId(selectedWalletId);
      const walletIds: Array<string | number> = [];
      for (const walletItem of nextWallets) {
        walletIds.push(walletItem.walletid);
      }
      const nextStakes = await fetchWalletStakes(walletIds, 'active');
      setStakeList(nextStakes);
    } finally {
      setIsLoading(false);
    }
  };

  void loadInitialData();
}, [user]);

const onWalletChange = (nextWalletId: string) => {
  setWalletId(nextWalletId);
};

const clearStatusMessage = () => {
  setStatusMessage({ type: '', message: '' });
};

const openStakeModal = (nextChainName: string) => {
  const validatorListForChain = stakeProvidersByChain[nextChainName] ?? [];
  setChainName(nextChainName);
  setValidatorId(validatorListForChain[0]?.id ?? '');
  setIsModalOpen(true);
};

const submitStake = async () => {
  if (isSavingStake) {
    return;
  }

  setIsSavingStake(true);

  try {
    const chainTicker =
      coinNameToTicker[chain!.name.toLowerCase()] ??
      chain!.name.slice(0, 5).toUpperCase();
    const nextApy = validator?.apy ?? chain!.staking.baseApy;
    const result = await placeStake(user, walletId, chainTicker, stakeUsd, nextApy);

    if (!result.ok) {
      setStatusMessage({
        type: 'Error',
        message: result.error ?? 'Failed to submit stake.',
      });
      return;
    }

    setIsModalOpen(false);
    setAmountText('10000');
    setPercentText('');
    setStatusMessage({ type: 'Success', message: 'Stake submitted and wallet balance updated.' });
    await loadPage(walletId);
  } finally {

```

```

        setIsSavingStake(false);
    }
};

const askSellStake = (stake: WalletStake, daysLeft: number) => {
    setSellStakeItem(stake);
    setSellDaysLeft(daysLeft);
};

const closeSellPopup = () => {
    if (sellingId !== null) {
        return;
    }
    setSellStakeItem(null);
    setSellDaysLeft(0);
};

const sellOneStake = async (stake: WalletStake, daysLeft: number) => {
    if (!user || sellingId !== null) {
        return;
    }

    setSellingId(stake.stakeid);
    try {
        const result = await sellStake(stake.stakeid, daysLeft);

        if (!result.ok) {
            setStatusMessage({
                type: 'Error',
                message: result.error ?? 'Failed to sell this stake.',
            });
            return;
        }

        if (result.isEarlySell) {
            setStatusMessage({
                type: 'Success',
                message: `Stake sold early. ${formatCurrency(result.amountBack ?? 0)} returned after
${formatCurrency(result.feeAmount ?? 0)} penalty.`,
            });
        } else {
            setStatusMessage({
                type: 'Success',
                message: `Stake sold. ${formatCurrency(result.amountBack ?? 0)} returned to your wallet.`,
            });
        }

        await loadPage(walletId);
        setSellStakeItem(null);
        setSellDaysLeft(0);
    } finally {
        setSellingId(null);
    }
};

const closeStakeModal = () => {
    setIsModalOpen(false);
};

const chainHealth = chain ? healthToStatus(chain.network.avgHealth) : null;
const healthView =
    chainHealth === 'healthy'
        ? healthMap.healthy
        : chainHealth === 'busy'
        ? healthMap.busy
        : chainHealth === 'congested'
        ? healthMap.congested
        : chainHealth === 'degraded'
        ? healthMap.degraded
        : healthMap.incident;

```

```
return (
  <div className="">
    <div className="max-w-6xl mx-auto space-y-6">
      <div className="p-6 md:p-8 grid gap-6 md:grid-cols-[1.2fr_1fr] md:items-end rounded-xl ">
        <div>
          <h1 className="mt-2 text-3xl md:text-4xl font-semibold">Stake</h1>
          <p className="mt-3 text-sm md:text-base max-w-2xl">
            Earn rewards on idle assets while helping secure major blockchain networks.
          </p>
          <div className="mt-4 inline-flex rounded-full border px-3 py-1 text-sm">
            <Tooltip text="Lock up your crypto to help secure the network and earn staking rewards in
return." position="bottom">
              What is staking?
            </Tooltip>
          </div>
        </div>
      </div>
    </div>

    <section className="grid gap-4 md:grid-cols-2">
      {CHAINS.map((chainItem) => {
        const ticker = coinNameToTicker[chainItem.name.toLowerCase()] ?? chainItem.name.slice(0,
3).toUpperCase();

        return (
          <div
            key={chainItem.name}
            className={`rounded-2xl border p-4 md:p-5 shadow-sm ${isDark ? 'border-gray-600' : 'border-
gray-200'}}`>
            >
              <div className="flex h-full flex-col justify-between gap-4">
                <div className="flex items-start justify-between gap-3">
                  <div className="flex items-center gap-3 min-w-0">
                    <img
                      src={`https://img.logokit.com/crypto/${ticker}?token=${logoApiKey}`}
                      alt={`${chainItem.name} logo`}
                      className="w-11 h-11 "
                    />
                    <div className="min-w-0">
                      <p className="font-semibold text-lg truncate">{chainItem.name}</p>
                      <p className="text-sm">
                        Lock-up {chainItem.staking.lockupDays}d · Unbond {chainItem.staking.unbondingDays}d
                      </p>
                    </div>
                  </div>
                </div>
                <div className="flex items-center justify-between gap-3">
                  <p className="text-xs">Rewards vary by validator and network conditions.</p>
                  <Button
                    type="button"
                    variant="primary"
                    size="sm"
                    className="!m-0 inline-flex flex-nowrap items-center gap-1 whitespace nowrap"
                    onClick={() => openStakeModal(chainItem.name)}
                  >
                    Stake
                  </Button>
                </div>
              </div>
            </div>
          );
        });
      })}
    </section>

    <section className={`rounded-2xl border p-4 md:p-5 ${isDark ? 'border-gray-600' : 'border-gray-200'}}`>
      <h3 className="text-2xl font-semibold">Active Stakes</h3>

      {isLoading ? (
        <p className="mt-3 text-sm">Loading active stakes...</p>
      ) : stakeList.length === 0 ? (
        <p className="mt-3 text-sm">No active stakes yet.</p>
```

```

    ) : (
      <div className="mt-4 grid gap-3">
        {stakeList.map((stake) => {
          const walletName =
            walletList.find((walletItem) => Number(walletItem.walletid) === Number(stake.walletid)).name
          ??
            `Wallet ${stake.walletid}`;
          const ticker = String(stake.ticker ?? '').toUpperCase();
          const lockupDays = getLockDays(stake.ticker);
          const remainingLockupDays = getDaysLeft(stake.createdat, lockupDays);
          const isSellingThisStake = sellingId === stake.stakeid;

          return (
            <div
              key={stake.stakeid}
              className={`w-full rounded-xl border p-4 text-left ${isDark ? 'border-gray-600' : 'border-
gray-200 bg-white'}}`>
              >
                <div className="flex flex-wrap items-center justify-between gap-4">
                  <div className="flex items-center gap-3 min-w-0">
                    <div className={`h-11 w-11 rounded-lg border overflow-hidden ${isDark ? 'border-gray-
500' : 'border-gray-300'}`>>
                      <img
                        src={`https://img.logokit.com/crypto/${ticker}?token=${logoApiKey}`}
                        alt={`${ticker} logo`}
                        className="h-full w-full object-cover"
                      />
                    </div>
                    <div className="min-w-0">
                      <p className="text-lg font-semibold leading-tight">{ticker}</p>
                      <p className="text-sm opacity-80 truncate">Wallet: {walletName}</p>
                    </div>
                  </div>

                  <div className="flex items-center gap-3 ml-auto">
                    <div className="text-right">
                      <p className="text-2xl font-semibold">${formatCurrency(stake.quantity)}</p>
                      <p className="text-sm opacity-80">APY {stake.apy}%</p>
                    </div>
                    <Button
                      type="button"
                      size="sm"
                      variant="red"
                      className="!m-0 min-w-[84px]"
                      onClick={() => askSellStake(stake, remainingLockupDays)}
                      disabled={sellingId !== null}
                    >
                      {isSellingThisStake ? 'Selling...' : 'Sell'}
                    </Button>
                  </div>
                </div>

                <div className="mt-3 flex flex-wrap items-center justify-between gap-3">
                  <p className={`text-sm font-medium ${remainingLockupDays > 0 ? 'text-gray-600' : 'text-
green-600'}`>>
                    {remainingLockupDays > 0 ? `${remainingLockupDays}d lock-up left` : 'Lock-up complete'}
                  </p>
                  <p className="text-sm opacity-80">Started {new
Date(stake.createdat).toLocaleDateString()}</p>
                </div>
              </div>
            );
          })}
        </div>
      )}
    </section>
  </div>

  {isModalOpen && chain && (
    <div className="stake-modal-overlay fixed inset-0 z-50 flex items-center overflow-auto justify-center
p-4">

```

```

<div className="stake-modal-panel w-full max-w-2xl rounded-3xl border p-6">
  <div className="flex items-start gap-4">
    <div>
      <h2 className="text-2xl font-semibold mt-1">Stake {chain.name}</h2>
    </div>
  </div>

  <div className="mt-5 grid gap-4 md:grid-cols-2">
    <div className="space-y-2">
      <label className="text-sm font-medium">Wallet</label>
      <select
        value={walletId}
        onChange={(event) => {
          void onWalletChange(event.target.value);
        }}
        className={inputClass}
        disabled={isLoading || isSavingStake}
      >
        {isLoading ? (
          <option value="">Loading wallets...</option>
        ) : walletList.length === 0 ? (
          <option value="">No wallets found</option>
        ) : (
          walletList.map((walletItem) => (
            <option key={walletItem.walletid} value={walletItem.walletid}>
              {walletItem.name} - ${formatCurrency(walletItem.usdt_balance ?? 0)}
            </option>
          ))
        )}
      </select>
      <p className="text-xs">Available USDT: ${formatCurrency(walletUsd)}</p>
    </div>

    <div className="space-y-2">
      <label className="text-sm font-medium">Validator</label>
      <select
        value={validator?.id ?? ''}
        onChange={(event) => setValidatorId(event.target.value)}
        className={inputClass}
        disabled={isSavingStake}
      >
        {validatorOptions.map((validatorItem) => (
          <option key={validatorItem.id} value={validatorItem.id}>
            {validatorItem.name} - {validatorItem.apy}% APY
          </option>
        ))}
      </select>
    </div>
  </div>

  <div className="stake-modal-subpanel mt-4 rounded-2xl border p-4 space-y-3">
    <div className="flex items-center justify-between text-sm">
      <span>Projected APY</span>
      <span className="font-semibold">{validator?.apy ?? chain.staking.baseApy}%</span>
    </div>

    <div className="flex items-center justify-between text-sm">
      <span>Network Status</span>
      <span className={`font-semibold ${healthView.className}`}>
        {healthView.label}
      </span>
    </div>

    <div className="flex items-center justify-between text-sm">
      <span>Unbonding Period</span>
      <span className="font-semibold">{chain.staking.unbondingDays}d</span>
    </div>
  </div>

  <div className="mt-4 space-y-2">
    <label className="text-sm font-medium">Amount (USD)</label>
  </div>

```

```

<div className="rounded-xl border px-3 py-2 flex items-center justify-between">
  <span className="text-xl font-semibold">${</span>
  <input
    type="number"
    min="0"
    step="0.01"
    value={amountText}
    onChange={(event) => onAmountChange(event.target.value)}
    className="w-full bg-transparent text-right text-xl font-semibold focus:outline-none"
    placeholder="0.00"
    disabled={isSavingStake}
  />
</div>
<div className="grid grid-cols-[1fr_auto] gap-2">
  <div className="rounded-xl border px-3 py-2 flex items-center gap-2">
    <input
      type="number"
      min="0"
      max="100"
      step="0.01"
      value={percentText}
      onChange={(event) => onPercentChange(event.target.value)}
      className="w-full bg-transparent text-sm font-medium focus:outline-none"
      placeholder="Percentage"
      disabled={isSavingStake}
    />
    <span className="text-sm font-semibold">{</span>
  </div>
  <div className="flex items-center gap-1">
    {[25, 50, 75, 100].map((percent) => (
      <button
        key={percent}
        type="button"
        className="stake-modal-subpanel !m-0 rounded-lg border px-2 py-1 text-xs"
        onClick={() => setPercent(percent)}
        disabled={isSavingStake}
      >
        {percent}%
      </button>
    )]}
  </div>
</div>

<div className="stake-modal-subpanel mt-4 flex items-center justify-between rounded-xl border px-3
py-2 text-sm">
  <div className="flex items-center gap-2">
    <AlertTriangle className="w-4 h-4" />
    <span>Lock-up: {chain.staking.lockupDays} days</span>
  </div>
  <Tooltip
    text={`You will not be able to access these funds for ${chain.staking.lockupDays} days after
staking.`}
    position="top"
  >
    Details
  </Tooltip>
</div>

<div className="mt-5 flex justify-end gap-3">
  <Button type="button" size="sm" variant="secondary" onClick={closeStakeModal} disabled=
{isSavingStake} className="!m-0">
    Close
  </Button>
  <Button
    type="button"
    variant="green"
    size="sm"
    onClick={submitStake}
    disabled={isSavingStake}
    className="!m-0"
  >

```

```

    >
      Stake
    </Button>
  </div>
</div>
</div>
)}

{sellStakeItem && (
  <div className="trade-confirm-overlay fixed inset-0 z-50 grid place-items-center px-4">
    <div className="trade-confirm-panel">
      <h2 className="text-lg font-semibold">Confirm Sell</h2>
      <p className="trade-confirm-copy mt-2 text-sm">
        {sellDaysLeft > 0
          ? `This stake still has ${sellDaysLeft} day(s) left. Selling now applies a 10% penalty.`
          : 'Sell this active stake and move funds back to your wallet?'}
      </p>
      <div className="trade-confirm-summary mt-4 rounded-lg p-3">
        <div className="flex items-center justify-between text-sm">
          <span>Asset</span>
          <span className="font-medium">{sellStakeItem.ticker}</span>
        </div>
        <div className="mt-2 flex items-center justify-between text-sm">
          <span>Amount</span>
          <span className="font-medium">${formatCurrency(sellStakeItem.quantity)}</span>
        </div>
        {sellDaysLeft > 0 && (
          <div className="mt-2 flex items-center justify-between text-sm">
            <span>Penalty</span>
            <span className="font-medium">10%</span>
          </div>
        )}
      </div>
      <div className="mt-5 flex justify-end gap-3">
        <Button
          type="button"
          size="sm"
          variant="secondary"
          onClick={closeSellPopup}
          disabled={sellingId !== null}
        >
          Cancel
        </Button>
        <Button
          type="button"
          size="sm"
          variant="red"
          onClick={() => {
            void sellOneStake(sellStakeItem, sellDaysLeft);
          }}
          disabled={sellingId !== null}
        >
          {sellingId !== null ? 'Selling...' : 'Confirm Sell'}
        </Button>
      </div>
    </div>
  </div>
)}

{statusMessage.message !== '' && (
  <div className="trade-confirm-overlay fixed inset-0 z-50 grid place-items-center px-4">
    <div className="trade-confirm-panel relative min-h-[10rem] ">
      <div className="flex items-center gap-3">
        <div>
          <h2
            className={`text-lg font-semibold ${
              statusMessage.type === 'Error' ? 'text-red-600' : 'text-green-600'
            }`}
          >
            <div>
              {statusMessage.type === 'Error' ? 'Stake Not Completed' : 'Stake Completed'}
            </div>
          </h2>

```

```
        </div>
    </div>

    <p className="trade-confirm-copy mt-4 text-sm">
        {statusMessage.message}
    </p>
    <Button
        type="button"
        onClick={clearStatusMessage}
        size="sm"
        variant="secondary"
        className="absolute bottom-3 right-3 rounded-full text-sm"
    >
        Close
    </Button>
</div>
</div>
    )}
</div>
    );
};

export default Stake;
```

FILE: src/pages/Trade.tsx

```

import { useEffect, useState } from 'react';
import { coinNameToTicker } from '../data/tickers';
import { TradingViewChart, type ChartMode } from '../Components/TradingChart';
import SearchBar from '../Components/SearchBar';
import { useFetchCoinMetrics } from '../utils/FetchCoinMetrics';
import { useFetchCryptoNews } from '../utils/FetchCryptoNews';
import NewsCard from '../Components/NewsCard';
import Button from '../Components/Button';
import Tooltip from '../Components/Tooltip';
import fetchWalletUSDTBalance from '../utils/FetchWalletUSDTBalance';
import { useAuth } from '../hooks/useAuth';
import placeMarketBuy from '../utils/actions/placeMarketBuy';
import placeMarketSell from '../utils/actions/placeMarketSell';
import fetchWalletHolding from '../utils/FetchWalletHolding';
import IncrementXP from '../utils/IncrementXP';
import fetchUserLevel, { type UserExperienceLevel } from '../utils/fetchUserLevel';

import type { wallet } from '../types/Wallet';

type Side = 'buy' | 'sell';
type NoticeType = '' | 'Success' | 'Error';
type TradePosition = {
  availableqty: number;
  avgcost: number;
};

const Trade = () => {
  const { user } = useAuth();
  // Basic trade form state.
  const [USD, setUSD] = useState(0);
  const [percentage, setPercentage] = useState('');
  const [coinName, setCoinName] = useState('bitcoin');
  const [side, setSide] = useState<Side>('buy');
  const logoToken = import.meta.env.VITE_LOGO_API;
  //Use coinNameToTicker get ticker for respective crypto
  const baseTicker = coinNameToTicker[coinName] ?? 'BTC';
  const [wallets, setWallets] = useState<wallet[]>([]);
  const [selectedWalletID, setSelectedWalletID] = useState('');
  const [loading, setLoading] = useState(false);
  const [messageText, setMessageText] = useState('');
  const [messageType, setMessageType] = useState<NoticeType>('');
  const [binanceLastPrice, setBinanceLastPrice] = useState<number | null>(null);
  const [binanceChange24h, setBinanceChange24h] = useState<number | null>(null);
  const [position, setPosition] = useState<TradePosition | null>(null);
  const [userLevel, setUserLevel] = useState<UserExperienceLevel>('beginner');
  const [chartMode, setChartMode] = useState<ChartMode>('line');

  useEffect(() => {
    // Load user wallets once we know who the user is.
    async function fetchWallets() {
      const nextWallets = await fetchWalletUSDTBalance(user);
      setWallets(nextWallets);
      if (nextWallets.length > 0) {
        setSelectedWalletID(String(nextWallets[0].walletid));
      } else {
        setSelectedWalletID('');
        setPosition(null);
      }
    }
    fetchWallets();
  }, [user]);

  useEffect(() => {
    // Fetch experience level (beginner/intermediate/advanced).
    fetchUserLevel(user).then((level) => setUserLevel(level));
  }, [user]);

  useEffect(() => {

```

```

    // Beginner mode forces line chart for clarity.
    if (userLevel === 'beginner') {
      setChartMode('line');
    }
  }, [userLevel]);

//Fetch coin metrics
const { data: metrics, error: metricsError } =
  useFetchCoinMetrics(coinName);
//Fetch crypto news
const { data: news, loading: newsLoading, error: newsError } =
  useFetchCryptoNews(coinName);

const showMessage = (type: Exclude<NoticeType, '>', text: string) => {
  setMessageType(type);
  setMessageText(text);
};

const clearMessage = () => {
  setMessageType('');
  setMessageText('');
};

const handleMarketDataChange = (marketData: {
  lastPrice: number | null;
  change24h: number | null;
}) => {
  // Price data comes from chart candles (Binance), not CoinGecko.
  setBinanceLastPrice(marketData.lastPrice);
  setBinanceChange24h(marketData.change24h);
};

const tradePrice = binanceLastPrice ?? 0;

const refreshWalletAndPosition = async () => {
  // Re-sync balances and holdings after a trade.
  const latestWallets = await fetchWalletUSDTBalance(user!);
  setWallets(latestWallets);

  // Use selected wallet if set, otherwise use the first wallet.
  const walletToUse = selectedWalletID || String(latestWallets[0]?.walletid || '');
  setSelectedWalletID(walletToUse);
  const latestPosition = await fetchWalletHolding(walletToUse, baseTicker);
  setPosition(latestPosition);
};

const syncAfterTrade = async () => {
  // Shared post-trade steps.
  await IncrementXP(user!, 10);
  await refreshWalletAndPosition();
};

const executeBuy = async () => {
  setLoading(true);

  try {
    const res = await placeMarketBuy(selectedWalletID, baseTicker, USD, tradePrice);

    if (!res) {
      showMessage('Error', 'Insufficient funds in wallet');
      return;
    }

    showMessage(
      'Success',
      `Successfully placed market buy for ${USD.toFixed(2)} of ${baseTicker} at ${tradePrice.toFixed(2)}`
    );
    await syncAfterTrade();
  } finally {

```

```

        setLoading(false);
    }
};
const executeSell = async () => {
    setLoading(true);

    try {
        const res = await placeMarketSell(
            selectedWalletID,
            baseTicker,
            USD / tradePrice,
            tradePrice
        );

        if (!res) {
            showMessage('Error', 'Insufficient asset quantity in wallet');
            return;
        }

        showMessage(
            'Success',
            `Successfully placed market sell for $$${USD.toFixed(2)} of ${baseTicker} at
            $$${tradePrice.toFixed(2)}`
        );
        await syncAfterTrade();
    } finally {
        setLoading(false);
    }
};

const handleBuy = (e: React.FormEvent) => {
    e.preventDefault();
    if (isInvalidTradeAmount) {
        return;
    }
    void executeBuy();
};
const handleSell = (e: React.FormEvent) => {
    e.preventDefault();
    if (isInvalidTradeAmount) {
        return;
    }
    void executeSell();
};

useEffect(() => {
    // Whenever wallet or selected coin changes, refresh the holding row.
    fetchWalletHolding(selectedWalletID, baseTicker)
        .then(setPosition);
}, [selectedWalletID, baseTicker]);

const activePosition = position && position.availableqty > 0 ? position : null;
const currentPrice = binanceLastPrice ?? (activePosition ? activePosition.avgcost : 0);

const positionValue = activePosition ? activePosition.availableqty * currentPrice : 0;
const unrealizedPnL = activePosition
    ? (currentPrice - activePosition.avgcost) * activePosition.availableqty
    : 0;
const pnlPercent = activePosition && activePosition.avgcost > 0
    ? ((currentPrice - activePosition.avgcost) / activePosition.avgcost) * 100
    : 0;

const selectedWallet = wallets.find((wallet) => String(wallet.walletid) === String(selectedWalletID));
const showBeginnerTooltips = userLevel === 'beginner';
const whatIfScenarios = [-30, 20, 50];
const showWhatIfBox = showBeginnerTooltips && activePosition;
let MaxTradeUSD = 0;
if (side === 'buy') {
    MaxTradeUSD = selectedWallet?.usdt_balance ?? 0;
} else if (activePosition) {
    MaxTradeUSD = activePosition.availableqty * currentPrice;
}

```

73/129

```

<Button
  type="button"
  size="sm"
  variant={chartMode === 'line' ? 'primary' : 'secondary'}
  className={!m-0}
  onClick={() => setChartMode('line')}
>
  Line
</Button>
<Button
  type="button"
  size="sm"
  variant={chartMode === 'candlestick' ? 'primary' : 'secondary'}
  className={!m-0}
  onClick={() => setChartMode('candlestick')}
>
  Candles
</Button>
</div>
)}
<TradingViewChart
  coin={coinName}
  mode={chartMode}
  onMarketDataChange={handleMarketDataChange}
/>

<div className="rounded-2xl p-4 md:p-5">
  {metricsError && (
    <p className="text-xs text-red-600 mb-2">
      Metrics unavailable for this asset: {metricsError}
    </p>
  )}
  <div className="grid grid-cols-2 md:grid-cols-4 gap-4 text-xs md:text-sm">
    <div>
      <p className="mb-1">
        {showBeginnerTooltips ? (
          <Tooltip text="Price multiplied by circulating supply.">Market cap</Tooltip>
        ) : (
          'Market cap'
        )}
      </p>
      <p className="font-semibold ">
        {metrics
          ? `$$${(metrics.marketCap / 10**9).toFixed(2)}B`
          : '-'}
      </p>
    </div>
    <div>
      <p className="mb-1">
        {showBeginnerTooltips ? (
          <Tooltip text="Total traded value over the last 24 hours.">24h volume</Tooltip>
        ) : (
          '24h volume'
        )}
      </p>
      <p className="font-semibold ">
        {metrics
          ? `$$${(metrics.volume24h / 10**9).toFixed(2)}B`
          : '-'}
      </p>
    </div>
    <div>
      <p className="mb-1">
        {showBeginnerTooltips ? (
          <Tooltip text="Value if max token supply were all in circulation.">FDV</Tooltip>
        ) : (
          'FDV'
        )}
      </p>
      <p className="font-semibold ">

```

```

    {metrics?.fullyDilutedValuation
      ? `$$${(metrics.fullyDilutedValuation / 10**9).toFixed(2)}B`
      : 'N/A'}
  </p>
</div>
<div>
  <p className="mb-1">
    {showBeginnerTooltips ? (
      <Tooltip text="Tokens currently available on the market.">Circulating Supply</Tooltip>
    ) : (
      'Circulating Supply'
    )}
  </p>
  <p className="font-semibold ">
    {metrics?.circulatingSupply
      ? `$$${(metrics.circulatingSupply / 10**9).toFixed(2)}B`
      : 'N/A'}
  </p>
</div>
</div>
</div>
<div className="rounded-2xl shadow-sm border border-[var(--border-color)] bg-[var(--surface-color)]
p-5 md:p-6">
  <h2 className="text-lg font-semibold mb-3 ">
    News
  </h2>

  {newsLoading ? (
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      {/*Shows a grid on screen with 3 coloumns on medium displays and above*/}
      {/* Iterates through a 3 indexed array and in each card displays "loading..." */}
      {[1, 2, 3].map((i) => (
        <div
          key={i}
          className="border rounded-xl border-[var(--border-color)] bg-[var(--muted-surface-color)]
h-48 flex items-center justify-center text-xs text-[var(--muted-text-color)] animate-pulse"
        >
          Loading...
        </div>
      )]}
    </div>
  ) : newsError ? (
    <div className="border rounded-xl border-[var(--border-color)] bg-[var(--muted-surface-color)] p-
4 text-sm text-[var(--muted-text-color)]">
      {/*Here, the error will be in red text*/}
      <p className="text-red-600 ">
        Unable to load news: {newsError}
      </p>
    </div>
  ) : (
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      {/* As in the loading state, it will return a grid with 3 columns*/}
      {/*The news will map over each of the 6 articles and pass the attributes of each article into
the news card component*/}
      {news.map((article) => (
        <NewsCard
          key={article.article_id}
          title={article.title}
          description={article.description || article.content}
          link={article.link}
          imageUrl={article.image_url}
          pubDate={article.pubDate}
          source={article.source_id}
        </>
      )]}
    </div>
  )}
</div>

```

```

</div>

{/* Right sidebar */}
<div className="lg:col-span-1">
  <div className="rounded-2xl shadow-sm border border-[var(--border-color)] bg-[var(--surface-color)]
p-3 sticky top-24">
    <div className = "w-full flex items-center justify-center mb-4">
      <h3 className = "text-3xl ">Spot</h3>
    </div>
    <div className="w-full border border-[var(--border-color)]" />
    <div className="flex mb-4 rounded-lg overflow-hidden ">
      <Button
        variant = {'${side === 'buy' ? 'green' : 'secondary' }}'
        size = "lg"
        type="button"
        onClick={() => setSide('buy')}
        className={'flex-1 ${
          side === 'buy'
            ? ' '
            : ' '
          }`}
      >
        Buy
      </Button>
      <Button
        size = "lg"
        variant = {'${side === 'sell' ? 'red' : 'secondary'}}'
        type="button"
        onClick={() => setSide('sell')}
        className={'flex-1 py-2 text-sm font-medium transition-all ${
          side === 'sell'
            ? ' '
            : ' '
          }`}
      >
        Sell
      </Button>
    </div>
    <div>
      <label className="block text-sm font-medium my-3 ">
        Select Wallet
      </label>
      <select
        value={selectedWalletID}
        onChange={(e) => setSelectedWalletID(String(e.target.value))}
        className="w-full px-4 py-2 border border-[var(--border-color)] rounded-md bg-[var(--
background-color)] text-[var(--text-color)]"
      >
        {wallets.map((wallet) => (
          <option key={wallet.walletid} value={String(wallet.walletid)}>
            {wallet.name} - ${wallet.usdt_balance?.toLocaleString()}
          </option>
        ))}
      </select>
    </div>
    <form onSubmit={side === 'buy' ? handleBuy : handleSell} className="space-y-4 mt-4">

      <div>
        <label className="block text-sm font-medium mb-2">
          {side === 'buy' ? 'Buy' : 'Sell'} Amount
        </label>
        <div className="border border-[var(--border-color)] rounded-lg p-4">
          <div className="flex flex-row space-x-10 justify-between w-full">
            <div className = "flex items-center ">
              <input
                type="number"
                step="any"
                value={USD || ''}
                onChange={(e) => setUSDFromAmount(parseFloat(e.target.value) || 0)}

```

```

        placeholder="0.0"
        className="text-2xl font-medium bg-transparent focus:outline-none w-24"
      />
    <span className="text-sm ">USD</span>
  </div>
  <div className = "flex items-center gap-2">
    <input
      type="number"
      step="any"
      value={currentPrice > 0 ? USD / currentPrice : 0}
      onChange={(e) => {
        // Calculate USD based on current asset price.
        const assetAmount = parseFloat(e.target.value) || 0;
        setUSDFromAmount(assetAmount * currentPrice);
      }}
      placeholder="0.00"
      className="text-lg text-[var(--muted-text-color)] bg-transparent focus:outline-none w-24"
    />
    <span className="text-sm font-medium ">
      {baseTicker}
    </span>
    <img
      src={`https://img.logokit.com/crypto/${baseTicker}?token=${logoToken}`}
      alt={baseTicker}
      className="w-6 h-6 rounded-full"
    />
  </div>
</div>
</div>

<div className="space-y-2">
  <div className="flex items-center justify-between">
    <label className="block text-sm font-medium">
      {side === 'buy' ? 'Wallet' : 'Position'} Percentage
    </label>
    <span className="text-xs text-[var(--muted-text-color)]">
      Available: ${MaxTradeUSD.toFixed(2)}
    </span>
  </div>
  <div className="grid grid-cols-4 gap-2">
    {[25, 50, 75, 100].map((preset) => (
      <Button
        key={preset}
        type="button"
        size="sm"
        variant={
          Number(percentage) === preset
            ? side === 'buy'
              ? 'green'
              : 'red'
            : 'secondary'
        }
        onClick={() => setUSDFromPercent(preset)}
        className="w-full"
      >
        {preset}%
      </Button>
    ))}
  </div>
  <div className=" rounded-lg p-3 flex items-center gap-2">
    <input
      type="number"
      min="0"
      max="100"
      step="0.01"
      value={percentage}
      onChange={(e) => setUSDFromPercent(parseFloat(e.target.value) || 0)}
      placeholder="0"
      className="text-lg font-medium bg-transparent focus:outline-none w-20"
    />
  </div>
</div>

```

```

        />
        <span className="text-sm">%</span>
      </div>
    </div>

    <div className="pt-2">
      <div className="flex justify-between text-xs text-[var(--muted-text-color)] mb-1">
        <span>Slippage:</span>
        <span className="font-medium">2%</span>
      </div>
    </div>

    {isInvalidTradeAmount ? (
<Button
  type="submit"
  variant={side === 'buy' ? 'green' : 'red'}
  size="lg"
  className="w-full"
  disabled
>
  Invalid {side === 'buy' ? 'Buy' : 'Sell'} Amount
</Button>
) : (
  <>
    {!loading ? (
      <Button
        type="submit"
        variant={side === 'buy' ? 'green' : 'red'}
        size="lg"
        className="w-full"

      >

        {side === 'buy' ? 'Total Buy Amount' : 'Total Sell Amount'}: $
        {USD.toFixed(2)}
      </Button>
    ) : (
      <Button
        type="submit"
        variant={side === 'buy' ? 'green' : 'red'}
        size="lg"
        className="w-full"
        disabled
      >
        Processing...
      </Button>
    )}
  </>
)}

</form>
    {activePosition && activePosition.availableqty > 0 && (
    <div className="mt-6 rounded-xl border border-[var(--border-color)] bg-[var(--muted-surface-color)] p-4
space-y-2">
      <h4 className="text-sm font-semibold">Position</h4>

      <div className="flex justify-between text-sm">
        <span>Quantity</span>
        <span>
          {activePosition.availableqty.toFixed(6)} {baseTicker}
        </span>
      </div>

      <div className="flex justify-between text-sm">
        <span>Avg Cost</span>
        <span>${activePosition.avgcost.toFixed(2)}</span>
      </div>

      <div className="flex justify-between text-sm">
        <span>Market Price</span>
        <span>${currentPrice.toFixed(2)}</span>
      </div>
    </div>
  )}

```

```

<div className="flex justify-between text-sm">
  <span>Position Value</span>
  <span>${positionValue.toFixed(2)}</span>
</div>

{showWhatIfBox && (
  <div className="rounded-lg border border-[var(--border-color)] bg-[var(--muted-surface-color)] p-3">
    <p className="text-xs font-semibold">What If</p>
    <div className="mt-2 space-y-1 text-xs">
      {whatIfScenarios.map((percent) => {
        const projectedBalance = positionValue * (1 + percent / 100);
        return (
          <div key={percent} className="flex items-center justify-between">
            <span>{percent > 0 ? `+${percent}%` : `-${percent}%`}</span>
            <span>${projectedBalance.toFixed(2)}</span>
          </div>
        );
      })}
    </div>
  </div>
)}

<div className="flex justify-between text-sm font-medium">
  <span>PnL</span>
  <span
    className={
      unrealizedPnL >= 0 ? 'text-green-600' : 'text-red-600'
    }
  >
    {unrealizedPnL >= 0 ? '+' : ''}
    ${unrealizedPnL.toFixed(2)} ({pnlPercent.toFixed(2)}%)
  </span>
</div>
</div>
)}

</div>
</div>

{messageText !== '' && (
  <div className="trade-confirm-overlay fixed inset-0 z-50 grid place-items-center px-4">
    <div
      className={`trade-confirm-panel relative min-h-[10rem] ${
        messageType === 'Error' ? 'ring-1 ring-red-300/70' : 'ring-1 ring-green-300/70'
      }`}
    >
      <div className="flex items-center gap-3">

        <div>
          <h2
            className={`text-lg font-semibold ${
              messageType === 'Error' ? 'text-red-600' : 'text-green-600'
            }`}
          >
            {messageType === 'Error' ? 'Order Not Completed' : 'Order Completed'}
          </h2>
        </div>
      </div>

      <p className="trade-confirm-copy mt-4 text-sm">
        {messageText}
      </p>
      <Button
        type="button"
        onClick={clearMessage}
        size = "sm"
        variant = "secondary"
        className="absolute bottom-3 right-3 rounded-full text-sm "
      >
        Close
      </Button>
    </div>
  </div>
)}

```

```
        </div>
      </div>
    )}

    </div>
  );
};

export default Trade;
```

FILE: src/pages/WalletDashboard.tsx

```

import { useCallback, useEffect, useState } from 'react';
import Button from '../Components/Button';
import { useAuth } from '../hooks/useAuth';
import { createWallet } from '../utils/CreateWallet';
import deleteWallet from '../utils/DeleteWallet';
import fetchWallets from '../utils/FetchWallets';
import fetchWalletAssets, { type WalletAssetPosition } from '../utils/FetchWalletAssets';
import fetchWalletStakes, { type WalletStake } from '../utils/FetchWalletStakes';
import fetchXp from '../utils/fetchXP';
import { type wallet } from '../types/Wallet';

type Notice = {
  type: 'Error' | 'Success' | '';
  message: string;
};

type InsightTone = 'good' | 'warning' | 'risk';

type WalletInsight = {
  tone: InsightTone;
  text: string;
};

type WalletEvaluation = {
  walletName: string;
  insights: WalletInsight[];
};

const normalizeTicker = (value: string) => value.trim().toUpperCase();

const evaluateWallet = (
  walletName: string,
  assetPositions: WalletAssetPosition[],
  walletStakes: WalletStake[],
): WalletEvaluation => {
  const totals: Record<string, number> = {};
  let totalValue = 0;
  let stableValue = 0;
  let stakedValue = 0;
  let topValue = 0;
  let cryptoCount = 0;
  const insights: WalletInsight[] = [];

  for (let i = 0; i < assetPositions.length; i += 1) {
    const position = assetPositions[i];
    const ticker = normalizeTicker(String(position.ticker || ''));
    const quantity = Number(position.availableqty) || 0;
    if (ticker === '' || quantity <= 0) continue;

    const price = ticker === 'USD' ? 1 : Number(position.avgcost) || 1;
    const value = quantity * price;
    if (value <= 0) continue;

    totals[ticker] = (totals[ticker] || 0) + value;
  }

  for (let i = 0; i < walletStakes.length; i += 1) {
    const stake = walletStakes[i];
    if (stake.status !== 'active') continue;
    const ticker = normalizeTicker(String(stake.ticker || ''));
    const value = (Number(stake.quantity) || 0) + (Number(stake.rewardsaccrued) || 0);
    if (ticker === '' || value <= 0) continue;

    stakedValue += value;
    totals[ticker] = (totals[ticker] || 0) + value;
  }

  for (const ticker in totals) {
    const value = totals[ticker];
  }

```

```

    totalValue += value;
    if (ticker === 'USDT') {
      stableValue += value;
    } else {
      cryptoCount += 1;
    }
    if (value > topValue) {
      topValue = value;
    }
  }

  if (totalValue <= 0) {
    insights.push({ tone: 'risk', text: 'No funded positions. Add funds to start.' });
    return { walletName, insights };
  }

  const stableRatio = stableValue / totalValue;
  const stakedRatio = stakedValue / totalValue;
  const topRatio = topValue / totalValue;

  if (stableRatio >= 0.7) {
    insights.push({ tone: 'good', text: 'Defensive wallet: mostly stable assets.' });
  } else if (stableRatio <= 0.2) {
    insights.push({ tone: 'warning', text: 'Higher-risk wallet: mostly crypto assets.' });
  } else {
    insights.push({ tone: 'good', text: 'Balanced mix of stable and crypto assets.' });
  }

  if (cryptoCount <= 1 || topRatio >= 0.6) {
    insights.push({ tone: 'warning', text: 'Concentration is high. One asset dominates.' });
  } else if (stakedRatio >= 0.5) {
    insights.push({ tone: 'warning', text: 'Large share is locked in staking.' });
  } else {
    insights.push({ tone: 'good', text: 'Structure looks healthy and diversified.' });
  }

  return { walletName, insights };
};

const WalletDashboard = () => {
  const { user, loading } = useAuth();
  const [xp, setXp] = useState<number>(0);
  const [wallets, setWallets] = useState<wallet[]>([]);
  const [isCreating, setIsCreating] = useState<boolean>(false);
  const [isSubmittingCreate, setIsSubmittingCreate] = useState<boolean>(false);
  const [walletName, setWalletName] = useState<string>('');
  const [deletingWalletId, setDeletingWalletId] = useState<string | null>(null);
  const [notice, setNotice] = useState<Notice>({ type: '', message: '' });

  const [selectedWalletForEvaluation, setSelectedWalletForEvaluation] = useState<wallet | null>(null);
  const [evaluationLoading, setEvaluationLoading] = useState(false);
  const [evaluationError, setEvaluationError] = useState('');
  const [walletEvaluation, setWalletEvaluation] = useState<WalletEvaluation | null>(null);

  const refreshData = useCallback(async () => {
    if (!user) {
      setXp(0);
      setWallets([]);
      return;
    }

    const [nextXp, nextWallets] = await Promise.all([fetchXp(user), fetchWallets(user)]);
    setXp(nextXp);
    setWallets(nextWallets);
  }, [user]);

  useEffect(() => {
    void refreshData();
  }, [refreshData]);

  const closeWalletEvaluation = () => {

```

```

    setSelectedWalletForEvaluation(null);
    setEvaluationLoading(false);
    setEvaluationError('');
    setWalletEvaluation(null);
  };

const openWalletEvaluation = async (walletItem: wallet) => {
  setSelectedWalletForEvaluation(walletItem);
  setEvaluationLoading(true);
  setEvaluationError('');
  setWalletEvaluation(null);

  try {
    const [positions, stakes] = await Promise.all([
      fetchWalletAssets(walletItem.walletid),
      fetchWalletStakes([walletItem.walletid, 'active'],
    ]);

    const nextEvaluation = evaluateWallet(walletItem.name, positions, stakes);
    setWalletEvaluation(nextEvaluation);
  } catch {
    setEvaluationError('Unable to evaluate this wallet right now. Please try again.');
```

```

  } finally {
    setEvaluationLoading(false);
  }
};

async function handleCreateWallet() {
  if (!user || !walletName.trim()) {
    return;
  }

  setIsSubmittingCreate(true);
  setNotice({ type: '', message: '' });

  try {
    await createWallet(user, walletName.trim());
    setWalletName('');
    setIsCreating(false);
    setNotice({ type: 'Success', message: 'Wallet created.' });
    await refreshData();
  } catch {
    setNotice({ type: 'Error', message: 'Failed to create wallet.' });
  } finally {
    setIsSubmittingCreate(false);
  }
}

async function handleDeleteWallet(walletItem: wallet) {
  if (!user) {
    return;
  }

  const confirmed = window.confirm(
    `Delete wallet "${walletItem.name}"? This will remove all wallet assets, trades and stakes.`
  );

  if (!confirmed) {
    return;
  }

  setDeletingWalletId(walletItem.walletid);
  setNotice({ type: '', message: '' });

  try {
    const result = await deleteWallet(user, walletItem.walletid);
    if (!result.ok) {
      setNotice({ type: 'Error', message: result.error ?? 'Failed to delete wallet.' });
      return;
    }
  }
}
```

```

    if (selectedWalletForEvaluation?.walletid === walletItem.walletid) {
      closeWalletEvaluation();
    }

    setNotice({ type: 'Success', message: 'Wallet deleted.' });
    await refreshData();
  } finally {
    setDeletingWalletId(null);
  }
}

if (loading) {
  return <div className="px-6 md:px-10 pb-10">Loading...</div>;
}

return (
  <div className="px-6 md:px-10 pb-10">
    <div className="max-w-5xl mx-auto space-y-6">
      <div className="flex flex-col gap-3 md:flex-row md:items-center md:justify-between">
        <h1 className="text-2xl md:text-3xl font-semibold">
          Hi, {user?.user_metadata.username}!
        </h1>
        <div className="flex items-center gap-3">
          <Button size="md" variant="green" onClick={() => setIsCreating(true)}>
            Create New Wallet
          </Button>
          <div className="rounded-xl border border-[var(--border-color)] px-4 py-2 text-sm">
            <span className="font-semibold">Total XP:</span>
            <span className="ml-1">{xp}</span>
          </div>
        </div>
      </div>
    </div>

    {notice.message !== '' && (
      <div
        className={`rounded-xl border px-4 py-3 text-sm ${
          notice.type === 'Error'
            ? 'border-red-300 text-red-700'
            : 'border-green-300 text-green-700'
        }`}
      >
        {notice.message}
      </div>
    )}

    <section className="rounded-2xl border border-[var(--border-color)] bg-[var(--surface-color)] p-4
space-y-3">
      <div className="flex items-center justify-between">
        <h2 className="text-lg font-semibold">Wallets</h2>
        <p className="text-xs text-[var(--muted-text-color)]">Click a wallet to run portfolio
evaluation</p>
      </div>

      {wallets.length === 0 ? (
        <div className="text-sm text-[var(--muted-text-color)]">
          You have no wallets yet. Create one to get started.
        </div>
      ) : (
        <div className="space-y-3">
          {wallets.map((walletItem) => (
            <div
              key={walletItem.walletid}
              className="rounded-lg border border-[var(--border-color)] p-4 flex flex-col gap-3 md:flex-row
md:items-center md:justify-between"
            >
              <div
                role="button"
                tabIndex={0}
                onClick={() => void openWalletEvaluation(walletItem)}
                onKeyDown={(event) => {
                  if (event.key === 'Enter' || event.key === ' ') {

```

85/129

```

    {!evaluationLoading && evaluationError === '' && walletEvaluation && (
      <section className="mt-6 rounded-xl border border-gray-300 bg-white p-4 space-y-3">
        <h3 className="text-lg font-semibold">Feedback</h3>
        {walletEvaluation.insights.map((insight, index) => {
          const rowClass =
            insight.tone === 'good'
              ? 'border-gray-300 bg-gray-50'
              : insight.tone === 'warning'
                ? 'border-gray-300 bg-gray-50'
                : 'border-gray-300 bg-gray-50';

          return (
            <div key={index} className={`rounded-md border p-3 text-sm ${rowClass}`}>
              {insight.text}
            </div>
          );
        })}
      </section>
    )}
  </div>
</div>
)}

{isCreating && (
  <div className="fixed inset-0 z-50 flex items-center justify-center bg-black/40 px-4">
    <div className="w-full max-w-sm rounded-xl border border-[var(--border-color)] bg-[var(--surface-color)] p-6 shadow-xl space-y-4">
      <h2 className="text-lg font-semibold">Create Wallet</h2>
      <input
        value={walletName}
        onChange={(e) => setWalletName(e.target.value)}
        placeholder="Wallet name"
        className="w-full rounded-lg border border-[var(--border-color)] bg-[var(--background-color)] px-
3 py-2 text-sm"
      />
      <div className="flex justify-end gap-3">
        <Button
          type="button"
          size="sm"
          variant="secondary"
          onClick={() => setIsCreating(false)}
          disabled={isSubmittingCreate}
        >
          Cancel
        </Button>
        <Button
          type="button"
          size="sm"
          variant="green"
          onClick={() => void handleCreateWallet()}
          disabled={!walletName.trim() || isSubmittingCreate}
        >
          {isSubmittingCreate ? 'Creating...' : 'Create'}
        </Button>
      </div>
    </div>
  </div>
)}
</div>
);
};

export default WalletDashboard;

```

FILE: src/types/Badge.ts

```
export type BadgeId =  
  | 'first_trade'  
  | 'xp_200'  
  | 'ten_trades'  
  | 'xp_500'  
  | 'xp_1000';  
  
export type BadgeDefinition = {  
  id: BadgeId;  
  symbol: string;  
  title: string;  
  description: string;  
  isUnlocked: (xp: number, tradeCount: number) => boolean;  
};
```

FILE: src/types/Binance.ts

```
export interface BinanceCandle {  
  0: number; // open time of candle  
  1: string; // open (open price of wick)  
  2: string; // high (Up wick)  
  3: string; // low (Bottom wick)  
  4: string; // close (close price of wick)  
  5: string; // volume (volume statistic)  
}
```

FILE: src/types/Button.ts

```
export default interface buttonProps{
  children: React.ReactNode,
  variant: string,
  className?: string,
  size: string,
  onClick?: () => void | Promise<void>,
  type?: "submit" | "button" | "reset" | undefined,
  disabled?: boolean
}
```

FILE: src/types/Wallet.ts

```
export type wallet = {  
  name: string;  
  walletid: string;  
  usdt_balance?: number;  
  staked_balance?: number;  
};
```

FILE: src/types/network.ts

```
export type NetworkState = {  
  score: number;  
  status: "healthy" | "busy" | "congested" | "degraded" | "incident";  
  congestion: number;  
  simulatedAt: number;  
};
```

FILE: src/types/stake.ts

```
export type ChainConfig = {
  id: string;
  name: string;
  symbol: string;

  staking: {
    baseApy: number;
    volatility: number;
    lockupDays: number;
    unbondingDays: number;
  };

  network: {
    avgHealth: number;
    volatility: number;
    incidentChance: number;
  };
};
```

FILE: src/types/stakeProvider.ts

```
import type { Validator } from './validator';

export type StakeProvider = Validator & {
  apy: number;
};
```

FILE: src/types/stakeUi.ts

```
export type StakeReceipt = {
  chainName: string;
  validatorName: string;
  walletName: string;
  amount: number;
  apy: number;
  lockupDays: number;
};

export type StakeMessage = {
  type: 'Error' | 'Success' | '';
  message: string;
};
```

FILE: src/types/validator.ts

```
export type Validator = {  
  id: string;  
  name: string;  
  commission: number;  
  uptimeModifier: number;  
  trustScore: number;  
};
```

FILE: src/utils/CreateWallet.ts

```
import { createClient, type User } from "@supabase/supabase-js";
import generateId from "../GenerateId";

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export const createWallet = async (user: User, name: string) => {
  // Create a new wallet entry in the 'tblwallets' table
  const id = generateId();
  const now = new Date().toISOString();

  const { error } = await supabase
    .from('tblwallets')
    .upsert({ walletid: id, user_id: user.id, name: name, createdate: now, updatedate: now })
  if (error) {
    console.error('Error creating wallet:', error);
    throw new Error('Failed to create wallet');
  }

  //Add 50XP
  const {error: xpError} = await supabase.rpc('incrementXP', {
    user_id: user.id,
    quantity: 50
  });

  if (xpError) {
    console.error('Error updating XP:', xpError);
    throw new Error('Failed to update XP');
  }
}
```

FILE: src/utils/DeleteWallet.ts

```
import { createClient, type User } from '@supabase/supabase-js';

type DeleteWalletResult = {
  ok: boolean;
  error?: string;
};

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export default async function deleteWallet(user: User | null, walletid: string): Promise<DeleteWalletResult> {
  if (!user) {
    return { ok: false, error: 'You must be logged in to delete wallets.' };
  }

  const numericWalletId = Number(walletid);
  const walletFilter = Number.isFinite(numericWalletId) ? numericWalletId : walletid;

  const walletCheck = await supabase
    .from('tblwallets')
    .select('walletid')
    .eq('walletid', walletFilter)
    .eq('user_id', user.id)
    .maybeSingle();

  if (walletCheck.error) {
    return { ok: false, error: 'Unable to verify wallet ownership.' };
  }

  if (!walletCheck.data) {
    return { ok: false, error: 'Wallet not found.' };
  }

  const tradeDelete = await supabase.from('tbltrades').delete().eq('walletid', walletFilter);
  if (tradeDelete.error) {
    return { ok: false, error: 'Unable to delete wallet trades.' };
  }

  const stakeDelete = await supabase.from('tblstakes').delete().eq('walletid', walletFilter);
  if (stakeDelete.error) {
    return { ok: false, error: 'Unable to delete wallet stakes.' };
  }

  const assetDelete = await supabase.from('tblwalletassets').delete().eq('walletid', walletFilter);
  if (assetDelete.error) {
    return { ok: false, error: 'Unable to delete wallet assets.' };
  }

  const walletDelete = await supabase
    .from('tblwallets')
    .delete()
    .eq('walletid', walletFilter)
    .eq('user_id', user.id);

  if (walletDelete.error) {
    return { ok: false, error: 'Unable to delete wallet.' };
  }

  return { ok: true };
}
```

FILE: src/utils/FetchCandleData.ts

```
import {
  type CandlestickData,
  type UTCTimestamp,
} from "lightweight-charts";
import { type BinanceCandle } from "../types/Binance";
import { coinNameToTicker } from "../data/tickers";
export async function fetchCandleData(
  name: string,
  interval = "1h",
  limit = 1000,
  endTime?: number
  //Type promise that returns in the format of an object with the candlestick data array
): Promise<CandlestickData<UTCTimestamp>[]> {
  //Use the object to retrieve the ticker and add the USDT ticker.
  const ticker = coinNameToTicker[name] + "USDT"
  //set url with the appropriate parameters
  let url = `https://api.binance.com/api/v1/klines?symbol=${ticker}&interval=${interval}&limit=${limit}`;
  if (endTime) url += `&endTime=${endTime}`;
  //fetch url

  const res = await fetch(url);
  const data: BinanceCandle[] = await res.json();
  //parseFloat converts the strings into floating point numbers and it maps through each array (binance
  candle) within the object
  return data.map(array => ({
    time: (array[0] / 1000) as UTCTimestamp,
    open: parseFloat(array[1]),
    high: parseFloat(array[2]),
    low: parseFloat(array[3]),
    close: parseFloat(array[4]),
  }));
}
```

FILE: src/utils/FetchCoinMetrics.ts

```
import { useEffect, useState } from "react";

interface CoinMetrics {
  price: number;
  marketCap: number;
  volume24h: number;
  circulatingSupply: number;
  totalSupply: number | null;
  change24h: number;
  fullyDilutedValuation: number | null;
}

export function useFetchCoinMetrics(coinId?: string) {
  // Update state to include the new metric
  const [data, setData] = useState<CoinMetrics | null>(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    if (!coinId) return;

    setLoading(true);
    setError(null);

    const url = `https://api.coingecko.com/api/v3/coins/${coinId}`;

    fetch(url)
      .then(res => {
        if (!res.ok) throw new Error(`CoinGecko request failed with status: ${res.status}. Check coin ID.`);
        return res.json();
      })
      .then(json => {
        const marketData = json.market_data;

        setData({
          price: marketData.current_price.usd,
          marketCap: marketData.market_cap.usd,
          volume24h: marketData.total_volume.usd,
          circulatingSupply: marketData.circulating_supply,
          fullyDilutedValuation: marketData.fully_diluted_valuation?.usd ?? null,
          totalSupply: marketData.total_supply ?? marketData.max_supply,
          change24h: marketData.price_change_percentage_24h,
        });
      })
      .catch(err => {
        console.error("Fetch Error:", err);
        setError(err.message);
      })
      .finally(() => setLoading(false));
  }, [coinId]);

  return { data, loading, error };
}
```

FILE: src/utils/FetchCryptoNews.ts

```

import { useEffect, useState } from "react";
import { coinNameToTicker } from "../data/tickers";

interface NewsArticle {
  article_id: string;
  title: string;
  link: string;
  keywords: string[] | null;
  creator: string[] | null;
  video_url: string | null;
  description: string | null;
  content: string | null;
  pubDate: string;
  image_url: string | null;
  source_id: string;
  source_priority: number;
  source_url: string;
  source_icon: string | null;
  language: string;
  country: string[];
  category: string[];
  ai_tag: string | null;
  sentiment: string | null;
  sentiment_stats: string | null;
  ai_region: string | null;
  ai_org: string | null;
  ai_person: string | null;
  ai_place: string | null;
  ai_exchange: string | null;
  ai_company: string | null;
  ai_product: string | null;
  ai_quote: string | null;
}

export function useFetchCryptoNews(coinName?: string) {
  const [data, setData] = useState<NewsArticle[]>([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
  //useEffect that is triggered each time the user searches for a new coin
  useEffect(() => {
    if (!coinName) {
      setData([]);
      return;
    }
    //Convert coin name to ticker
    const newsCode = coinNameToTicker[coinName.toLowerCase()];
    if (!newsCode) {
      setError(`No news code found for coin: ${coinName}`);
      setData([]);
      return;
    }

    setLoading(true);
    setError(null);
    const apiKey = import.meta.env.VITE_NEWSDATA_API
    // Free tier may not support from_date parameter, so we'll try without it first
    // If needed, you can add from_date for paid tiers
    const url = `https://newsdata.io/api/1/crypto?apikey=${apiKey}&coin=${newsCode.toLowerCase()}`;

    fetch(url)
      .then(async (res) => {
        const json = await res.json();
        //Set data as the first 6
        setData(json.results.slice(0, 6));
      })
    //Catch the error
  }, [coinName]);
}

```

```
        .catch((err) => {
            console.error("News Fetch Error:", err);
            setError(err.message);
            setData([]);
        })
        //Set loading to false
        .finally(() => setLoading(false));
    }, [coinName]);
    //Return data, loading and error
    return { data, loading, error };
}
```

FILE: src/utils/FetchWalletAssets.ts

```
import { createClient } from '@supabase/supabase-js';

export type WalletAssetPosition = {
  walletid: number;
  ticker: string;
  availableqty: number;
  avgcost: number;
  stakedqty: number;
};

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!,
);

export default async function fetchWalletAssets(walletId: string | number): Promise<WalletAssetPosition[]> {
  const normalizedWalletId = Number(walletId);
  if (!Number.isFinite(normalizedWalletId)) {
    return [];
  }

  const { data, error } = await supabase
    .from('tblwalletassets')
    .select('walletid,ticker,availableqty,avgcost,StakedQty')
    .eq('walletid', normalizedWalletId);

  if (error) {
    console.error('Error fetching wallet assets:', error);
    return [];
  }

  return (data ?? []).map((asset) => ({
    walletid: Number(asset.walletid ?? normalizedWalletId),
    ticker: String(asset.ticker ?? '').toUpperCase(),
    availableqty: Number(asset.availableqty ?? 0),
    avgcost: Number(asset.avgcost ?? 0),
    stakedqty: Number(asset.StakedQty ?? 0),
  }));
}
```

FILE: src/utils/FetchWalletHolding.ts

```
import { createClient } from '@supabase/supabase-js';

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export default async function fetchWalletHolding(
  walletid: string,
  ticker: string
): Promise<{ availableqty: number; avgcost: number } | null> {
  const { data, error } = await supabase
    .from('tblwalletassets')
    .select('availableqty, avgcost')
    .eq('walletid', walletid)
    .eq('ticker', ticker)
    .single();

  if (error) return null;
  return data;
}
```

FILE: src/utils/FetchWalletStakes.ts

```
import { createClient } from '@supabase/supabase-js';

export type WalletStake = {
  stakeid: number;
  walletid: number;
  ticker: string;
  quantity: number;
  apy: number;
  status: 'active' | 'completed' | 'early_unstaked';
  rewardsaccrued: number;
  penaltyapplied: number;
  createdat: string;
};

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export default async function fetchWalletStakes(
  walletIds: Array<string | number>,
  status: WalletStake['status'],
): Promise<WalletStake[]> {
  if (walletIds.length === 0) {
    return [];
  }

  if (status){
    //Supabase fetch, order stakes in date of creation, where status is equal to the status from the frontend.
    const { data, error } = await supabase
      .from('tblstakes')
      .select('stakeid,walletid,ticker,quantity,apy,status,rewardsaccrued,penaltyapplied,createdat')
      .in('walletid', walletIds)
      .eq('status', status)
      .order('createdat', { ascending: false });

    if (error) {
      console.error('Error fetching wallet stakes:', error);
      return [];
    }

    const stakes: WalletStake[] = [];
    for (const stake of data ) {
      //push the stakes to the stakes array and if a field doesn't exist push 0 or '' instead.
      stakes.push({
        stakeid: Number(stake.stakeid ?? 0),
        walletid: Number(stake.walletid ?? 0),
        ticker: String(stake.ticker ?? ''),
        quantity: Number(stake.quantity ?? 0),
        apy: Number(stake.apy ?? 0),
        status: (stake.status ?? 'active'),
        rewardsaccrued: Number(stake.rewardsaccrued ?? 0),
        penaltyapplied: Number(stake.penaltyapplied ?? 0),
        createdat: String(stake.createdat ?? ''),
      });
    }

    return stakes;
  }
  else {
    return []
  }
}
```

FILE: src/utils/FetchWalletUSDTBalance.ts

```
import { createClient, type User } from '@supabase/supabase-js'
import { type wallet } from '../types/Wallet';
import fetchWallets from './FetchWallets.js';
const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export default async function fetchWalletUSDTBalance(
  user: User | null
  //Promise that returns an array of wallets
): Promise<wallet[]> {
  const wallets = await fetchWallets(user);
  if (!user) return wallets;
  for (const wallet of wallets) {
    const { data, error } = await supabase
      .from('tblwalletassets')
      .select('availableqty,StakedQty')
      .eq('walletid', wallet.walletid)
      .eq('ticker', 'USDT')
      .single();
    if (error) {
      console.error(`Error fetching USDT balance for wallet ${wallet.walletid}:`, error);
      wallet['usdt_balance'] = 0;
      wallet['staked_balance'] = 0;
    } else {
      wallet['usdt_balance'] = Number(data.availableqty ?? 0);
      wallet['staked_balance'] = Number(data.StakedQty ?? 0);
    }
  }
  return wallets;
}
```

FILE: src/utils/FetchWallets.ts

```
import { createClient, type User } from '@supabase/supabase-js'
import { type wallet } from '../types/Wallet';
const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export default async function fetchWallets(
  user: User | null
  //Promise that returns an array of wallets
): Promise<wallet[]> {
  if (!user) return [];

  const { data, error } = await supabase
    .from('tblwallets')
    .select('walletid, name')
    .eq('user_id', user.id);

  if (error) {
    console.error('Error fetching wallets:', error);
    return [];
  }

  return data ?? [];
}
```

FILE: src/Utils/GenerateId.ts

```
export default function generateId(): number {  
  return Math.floor(Math.random() * 90000000 + 10000000);  
}
```

FILE: src/utils/IncrementXP.ts

```
import { createClient, type User } from '@supabase/supabase-js'

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export default async function incrementXp(user: User | null, quantity: number): Promise<void> {
  if (!user) return;

  const {error: xpError} = await supabase.rpc('incrementXP', {
    user_id: user.id,
    quantity: quantity
  });
  if (xpError) {
    console.error('Error checking onboarding status:', xpError);
  }
}
```

FILE: src/utils/UpdateSkillLevel.ts

```
import { createClient, type User } from '@supabase/supabase-js'

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export default async function updateSkillLevel(user: User | null, skillLevel: string): Promise<boolean> {
  if (!user) return false;

  // Update the user's onboarded status to true
  let { error } = await supabase
    .from('tblusers')
    .update({ onboarded: true })
    .eq('id', user.id);
  if (error) {
    console.error('Error checking onboarding status:', error);
    return false;
  }
  // Update the user's skill level
  ({ error } = await supabase
    .from('tblusers')
    .update({ userlevel: skillLevel })
    .eq('id', user.id)
  );
  if (error) {
    console.error('Error updating skill level:', error);
    return false;
  }
  return true;
}
```

FILE: src/utils/actions/placeMarketBuy.ts

```

import { createClient } from '@supabase/supabase-js'
import generateId from '../GenerateId';
const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export default async function placeMarketBuy(
  walletid: string,
  ticker: string,
  USD: number,
  price: number,
): Promise<boolean> {
  //Input walletid and ticker and quantity to buy
  //Use fetch wallets function to find t
  //Insert a new row into tbltrades with wallet id, ticker, quantity, side 'buy' and price
  //Use database logic to add or update a position
  const availableUSD= await supabase
    .from('tblwalletassets')
    .select('availableqty')
    .eq('walletid', walletid)
    .eq('ticker', 'USDT')
    .single();

  if (availableUSD.error) {
    console.error('Error fetching wallet USD balance:', availableUSD.error);
    return false;
  }
  if (availableUSD.data.availableqty < USD) {
    console.error('Insufficient USD balance in wallet');
    return false;
  }
  const newUSDBalance = availableUSD.data.availableqty - USD;
  const { error: updateError } = await supabase
    .from('tblwalletassets')
    .update({ availableqty: newUSDBalance, updatedat: new Date() })
    .eq('walletid', walletid)
    .eq('ticker', 'USDT');
  if (updateError) {
    console.error('Error updating wallet USD balance:', updateError);
    return false;
  }

  const quantityToBuy = USD / price;
  // Deduct a 1% fee from the USD value
  const fee = 0.01 * USD;
  USD -= fee;
  // No slippage for now
  const slippage = 0;
  const id = generateId();
  const { error } = await supabase
    .from('tbltrades')
    .insert([
      {
        tradeid: id,
        walletid: walletid,
        ticker: ticker,
        usdvalue: USD,
        quantity: quantityToBuy,
        side: 'BUY',
        price: price,
        fee: fee,
        slippage: slippage,
        createdate: new Date().toISOString(),
      },
    ]);
  if (error) {
    console.error('Error placing market buy:', error);
  }
}

```

```
    }  
    const {error: incrementError }=  await supabase.rpc('increment_wallet_asset', {  
      p_price: price,  
      p_walletid: walletid,  
      p_ticker: ticker,  
      p_qty: quantityToBuy,  
    });  
    if (incrementError) {  
      console.error('Error updating wallet asset quantity:', incrementError);  
      return false;  
    }  
  
    return true  
  }  
}
```

FILE: src/utils/actions/placeMarketSell.ts

```
import { createClient } from '@supabase/supabase-js'
import generateId from '../GenerateId';

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export default async function placeMarketSell(
  walletid: string,
  ticker: string,
  quantityToSell: number,
  price: number,
): Promise<boolean> {
  // Check asset balance
  const asset = await supabase
    .from('tblwalletassets')
    .select('availableqty, avgcost')
    .eq('walletid', walletid)
    .eq('ticker', ticker)
    .single();

  if (asset.error) {
    console.error('Error fetching asset balance:', asset.error);
    return false;
  }
  if (asset.data.availableqty < quantityToSell) {
    console.error('Insufficient asset quantity');
    return false;
  }

  // Calculate USDT value, fee, and net USDT. Slippage is 0 for now.
  const grossUSD = quantityToSell * price;
  const fee = 0.01 * grossUSD;
  const netUSD = grossUSD - fee;
  const slippage = 0;

  // Sell the asset
  const { error: updateAssetError } = await supabase
    .from('tblwalletassets')
    .update({
      availableqty: asset.data.availableqty - quantityToSell,
      updatedat: new Date()
    })
    .eq('walletid', walletid)
    .eq('ticker', ticker);

  if (updateAssetError) {
    console.error('Error updating asset quantity:', updateAssetError);
    return false;
  }

  // Increase USDT balance
  const usd = await supabase
    .from('tblwalletassets')
    .select('availableqty')
    .eq('walletid', walletid)
    .eq('ticker', 'USDT')
    .single();

  if (usd.error) {
    console.error('Error fetching USD balance:', usd.error);
    return false;
  }

  const { error: updateUsdError } = await supabase
    .from('tblwalletassets')
    .update({
      availableqty: usd.data.availableqty + netUSD,
```

```
        updatedat: new Date()
    })
    .eq('walletid', walletid)
    .eq('ticker', 'USDT');

if (updateUsdError) {
    console.error('Error updating USD balance:', updateUsdError);
    return false;
}

// Insert trade
const tradeId = generateId();
const { error: tradeError } = await supabase
    .from('tbltrades')
    .insert([
        {
            tradeid: tradeId,
            walletid,
            ticker,
            usdvalue: netUSD,
            quantity: quantityToSell,
            side: 'SELL',
            price,
            fee,
            slippage,
            createdate: new Date().toISOString(),
        }
    ]);

if (tradeError) {
    console.error('Error inserting sell trade:', tradeError);
    return false;
}

return true;
}
```

FILE: src/utils/actions/placeStake.ts

```

import { createClient, type User } from '@supabase/supabase-js';
import generateId from '../GenerateId';

type PlaceStakeResult = {
  ok: boolean;
  error?: string;
};

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export default async function placeStake(
  user: User | null,
  walletid: string,
  ticker: string,
  amountUSD: number,
  apy: number,
): Promise<PlaceStakeResult> {
  if (!user) {
    return { ok: false, error: 'You must be logged in to stake.' };
  }

  if (amountUSD <= 0) {
    return { ok: false, error: 'Enter a valid stake amount.' };
  }

  const normalizedWalletId = Number(walletid);

  // Get current USDT balance for this wallet.
  const usdBalanceRow = await supabase
    .from('tblwalletassets')
    .select('availableqty, StakedQty')
    .eq('walletid', normalizedWalletId)
    .eq('ticker', 'USDT')
    .single();

  if (usdBalanceRow.error || !usdBalanceRow.data) {
    return { ok: false, error: 'Unable to fetch wallet USDT balance.' };
  }
  //Convert to number
  const availableUSDT = Number(usdBalanceRow.data.availableqty ?? 0);
  const stakedUSDT = Number(usdBalanceRow.data.StakedQty ?? 0);

  if (availableUSDT < amountUSD) {
    return { ok: false, error: 'Insufficient USDT balance in this wallet.' };
  }

  const { error: updateWalletError } = await supabase
    .from('tblwalletassets')
    .update({
      availableqty: availableUSDT - amountUSD,
      StakedQty: stakedUSDT + amountUSD,
      updatedat: new Date(),
    })
    .eq('walletid', normalizedWalletId)
    .eq('ticker', 'USDT');

  if (updateWalletError) {
    return { ok: false, error: 'Failed to update wallet balance.' };
  }

  // Save a simple stake record.
  const stakeId = generateId();
  const { error: insertStakeError } = await supabase
    .from('tblstakes')
    .insert([
      {

```

```
        stakeid: stakeId,  
        walletid: normalizedWalletId,  
        ticker,  
        quantity: amountUSD,  
        apy,  
        status: 'active',  
        rewardsaccrued: 0,  
        penaltyapplied: 0,  
        createdat: new Date().toISOString(),  
    },  
    ]);  
  
    if (insertStakeError) {  
        return { ok: false, error: 'Failed to save stake record.' };  
    }  
  
    return { ok: true };  
}
```

FILE: src/utils/actions/sellStake.ts

```
import { createClient } from '@supabase/supabase-js';

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export default async function sellStake(
  stakeId: number,
  daysLeft: number,
){
  const { data: stakeRow, error: stakeError } = await supabase
    .from('tblstakes')
    .select('walletid,quantity')
    .eq('stakeid', stakeId)
    .single();

  if (stakeError || !stakeRow) {
    return { ok: false, error: 'Unable to fetch stake details.' };
  }

  const walletId = stakeRow.walletid;
  const stakeAmount = stakeRow.quantity ;

  const { data: walletRow, error: walletError } = await supabase
    .from('tblwalletassets')
    .select('availableqty, StakedQty')
    .eq('walletid', walletId)
    .eq('ticker', 'USDT')
    .single();

  if (walletError || !walletRow) {
    return { ok: false, error: 'Unable to fetch wallet balances.' };
  }

  const availableUsd = walletRow.availableqty
  const stakedUsd = walletRow.StakedQty ;

  const isEarlySell = daysLeft > 0;
  const feeRate = isEarlySell ? 0.1 : 0;
  const feeAmount = stakeAmount * feeRate;
  const amountBack = stakeAmount - feeAmount;

  const { error: walletUpdateError } = await supabase
    .from('tblwalletassets')
    .update({
      availableqty: availableUsd + amountBack,
      StakedQty: stakedUsd - stakeAmount,
      updatedat: new Date(),
    })
    .eq('walletid', walletId)
    .eq('ticker', 'USDT');

  if (walletUpdateError) {
    return { ok: false, error: 'Failed to release staked funds into wallet balance.' };
  }

  const { error: stakeUpdateError } = await supabase
    .from('tblstakes')
    .update({
      status: isEarlySell ? 'early_unstaked' : 'completed',
      penaltyapplied: feeAmount,
    })
    .eq('stakeid', stakeId);

  if (stakeUpdateError) {
    return { ok: false, error: 'Failed to mark stake as sold.' };
  }
}
```

```
    }  
  
    return {  
      ok: true,  
      isEarlySell,  
      feeAmount,  
      amountBack,  
    };  
  }  
}
```

FILE: src/utils/fetchUserLevel.ts

```
import type { User } from '@supabase/supabase-js';
import { supabase } from '../lib/supabaseClient';

export type UserExperienceLevel = 'beginner' | 'advanced';

const mapUserLevel = (value: string | null | undefined): UserExperienceLevel => {
  const normalized = (value ?? '').toLowerCase();

  if (normalized === 'beginner') {
    return 'beginner';
  }

  if (normalized === 'intermediate' || normalized === 'advanced') {
    return 'advanced';
  }

  return 'beginner';
};

export default async function fetchUserLevel(user: User | null): Promise<UserExperienceLevel> {
  if (!user) {
    return 'beginner';
  }

  const { data, error } = await supabase
    .from('tblusers')
    .select('userlevel')
    .eq('id', user.id)
    .single();

  if (error) {
    return 'beginner';
  }

  return mapUserLevel(data?.userlevel);
}
```

FILE: src/utils/fetchXP.ts

```
import { createClient, type User } from '@supabase/supabase-js'

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export default async function fetchXp(user: User | null): Promise<number> {
  if (!user) return 0;

  const { data, error } = await supabase
    .from('tblusers')
    .select('xp')
    .eq('id', user.id)
    .single();

  if (error) {
    console.error('Error checking onboarding status:', error);
    return 0;
  }

  return data.xp;
}
```

FILE: src/utils/leaderboardUtils.ts

```
import { supabase } from '../lib/supabaseClient';

const XP_PER_TRADE = 10;
const XP_PER_WALLET = 50;
const WALLET_STARTING_USDT = 100000;
const key = (value: unknown) => String(value ?? '').trim().toLowerCase();

export type LeaderboardUser = {
  userId: string;
  username: string | null;
  email: string | null;
  xpAllTime: number | null;
  xp7d: number;
  portfolioAllTime: number;
  portfolio7d: number;
  portfolioAllTimeWalletName: string;
  portfolio7dWalletName: string;
};

export type LeaderboardRow = LeaderboardUser & {
  score: number;
  rank: number;
};

export const buildLeaderboardRows = (
  leaderboardUsers: LeaderboardUser[],
  selectedMetric: 'xp' | 'PnL',
  selectedTimeframe: '7d' | 'all-time',
) => {
  // This list will hold users with one common score field,
  // so UI rendering can use the same table for XP and Portfolio modes.
  const rankedRows: LeaderboardRow[] = [];

  // Step 1: pick the correct score for each user based on the active filters.
  // - XP + 7D      -> xp7d
  // - XP + All-time -> xpAllTime
  // - Portfolio + 7D -> portfolio7d
  // - Portfolio + All-time -> portfolioAllTime
  for (let i = 0; i < leaderboardUsers.length; i += 1) {
    const leaderboardUser = leaderboardUsers[i];
    let selectedScore = 0;

    if (selectedMetric === 'xp') {
      if (selectedTimeframe === '7d') {
        selectedScore = leaderboardUser.xp7d || 0;
      } else {
        selectedScore = leaderboardUser.xpAllTime || 0;
      }
    } else if (selectedMetric === 'PnL') {
      if (selectedTimeframe === '7d') {
        selectedScore = leaderboardUser.portfolio7d || 0;
      } else {
        selectedScore = leaderboardUser.portfolioAllTime || 0;
      }
    }

    // Keep all original user fields and attach the computed score used for ranking.
    rankedRows.push({
      ...leaderboardUser,
      score: selectedScore,
      rank: 0,
    });
  }

  // Step 2: sort highest score first.
  // If two users have the same score, sort by username alphabetically for stable order.
  rankedRows.sort((leftRow, rightRow) => {
    if (rightRow.score !== leftRow.score) {
      return rightRow.score - leftRow.score;
    }
    return String(leftRow.username).localeCompare(String(rightRow.username));
  });
}
```

```

    });

    // Step 3: assign rank numbers after sorting (1-based rank).
    for (let i = 0; i < rankedRows.length; i += 1) {
        rankedRows[i].rank = i + 1;
    }

    // Final output consumed by Leaderboard.tsx.
    return rankedRows;
};

type DbUserRow = {
    id: unknown;
    username: unknown;
    email: unknown;
    xp: unknown;
};

type DbWalletRow = {
    walletid: unknown;
    user_id: unknown;
    name: unknown;
    createdate: unknown;
};

type DbAssetRow = {
    walletid: unknown;
    ticker: unknown;
    availableqty: unknown;
};

type DbStakeRow = {
    walletid: unknown;
    quantity: unknown;
    rewardsaccrued: unknown;
    status: unknown;
    createdat: unknown;
};

type DbTradeRow = {
    walletid: unknown;
    ticker: unknown;
    side: unknown;
    quantity: unknown;
    usdvalue: unknown;
    fee: unknown;
    createdate: unknown;
};

type WalletInfo = Record<string, { userId: string; walletName: string }>;
type HoldingsByWallet = Record<string, Record<string, number>>;

export default async function fetchLeaderboardData(): Promise<LeaderboardUser[]> {
    // Build a cutoff for "last 7 days" logic used in trades and wallet-created XP.
    const now = Date.now();
    const sevenDaysMs = 7 * 24 * 60 * 60 * 1000;
    const cutoffMs = now - sevenDaysMs;
    // Fetch all users
    const usersResponse = await supabase.from('tblusers').select('id,username,email,xp');
    if (usersResponse.error) {
        throw new Error(usersResponse.error.message);
    }
    //Fetch all wallets
    const walletsResponse = await supabase.from('tblwallets').select('walletid,user_id,name,createdate');
    if (walletsResponse.error) {
        throw new Error(walletsResponse.error.message);
    }
    // Fetch all assets
    const assetsResponse = await supabase.from('tblwalletassets').select('walletid,ticker,availableqty');
    if (assetsResponse.error) {
        throw new Error(assetsResponse.error.message);
    }
}

```

```

    }
    //Fetch all stakes
    const stakesResponse = await
supabase.from('tblstakes').select('walletid,quantity,rewardsaccrued,status,createdat');
    if (stakesResponse.error) {
        throw new Error(stakesResponse.error.message);
    }
    //Fetch all trades
    const tradesResponse = await supabase
        .from('tbltrades')
        .select('walletid,ticker,side,quantity,usdvalue,fee,createdate');
    if (tradesResponse.error) {
        throw new Error(tradesResponse.error.message);
    }

    // Normalize null responses to empty arrays so loops can run safely.
    const users = (usersResponse.data || []) as DbUserRow[];
    const wallets = (walletsResponse.data || []) as DbWalletRow[];
    const assets = (assetsResponse.data || []) as DbAssetRow[];
    const stakes = (stakesResponse.data || []) as DbStakeRow[];
    const allTrades = (tradesResponse.data || []) as DbTradeRow[];

    const trades: DbTradeRow[] = [];
    for (let i = 0; i < allTrades.length; i += 1) {
        const trade = allTrades[i];
        const createdMs = new Date(String(trade.createdate)).getTime();
        if (createdMs >= cutoffMs) {
            trades.push(trade);
        }
    }

    // walletInfo maps walletId -> owner user + display name.
    // currentHoldings maps walletId -> { ticker: qty } for "current" balances.
    const walletInfo: WalletInfo = {};
    const currentHoldings: HoldingsByWallet = {};

    // Build wallet lookup and initialize empty holdings buckets.
    for (let i = 0; i < wallets.length; i += 1) {
        const wallet = wallets[i];
        const walletId = String(wallet.walletid);
        const walletName = String(wallet.name || '').trim() || `Wallet ${walletId}`;

        walletInfo[walletId] = {
            userId: key(wallet.user_id),
            walletName,
        };

        currentHoldings[walletId] = {};
    }

    // Fill currentHoldings from wallet asset rows.
    for (let i = 0; i < assets.length; i += 1) {
        const asset = assets[i];
        const walletId = String(asset.walletid);
        if (!walletInfo[walletId]) continue;
        const ticker = String(asset.ticker);
        const qty = Number(asset.availableqty) || 0;
        currentHoldings[walletId][ticker] = (Number(currentHoldings[walletId][ticker]) || 0) + qty;
    }

    // Create a separate snapshot for historical balances (7d ago estimate).
    // We reverse recent trades on this snapshot only.
    const historicalHoldings: HoldingsByWallet = structuredClone(currentHoldings);

    // Tracks trade count per user for XP(7d) calculation.
    const tradeCountByUser: Record<string, number> = {};

    // Reverse each recent trade to estimate balances at cutoff time.
    for (let i = 0; i < trades.length; i += 1) {
        // Read each trade row and locate the wallet it belongs to.

```

```

const trade = trades[i];
const walletId = String(trade.walletid);
const info = walletInfo[walletId];
if (!info) continue;

// Ticker & user are the keys used to update holdings and XP counters.
const ticker = String(trade.ticker || '').trim().toUpperCase();
// Parse the trade values used in reverse math.
const quantity = Number(trade.quantity) || 0;
const usdValue = Number(trade.usdvalue) || 0;
const fee = Number(trade.fee) || 0;
const side = String(trade.side || '').trim().toUpperCase();

if (historicalHoldings[walletId][ticker] === undefined) {
  historicalHoldings[walletId][ticker] = 0;
}
if (historicalHoldings[walletId].USDT === undefined) {
  historicalHoldings[walletId].USDT = 0;
}

// Reverse BUY trade:
// remove bought asset from historical snapshot and add spent USDT back.
if (side === 'BUY') {
  historicalHoldings[walletId][ticker] = historicalHoldings[walletId][ticker] - quantity;
  historicalHoldings[walletId].USDT = historicalHoldings[walletId].USDT + usdValue + fee;
}
// Reverse SELL trade:
// add sold asset back and remove received USDT.
if (side === 'SELL') {
  historicalHoldings[walletId][ticker] = historicalHoldings[walletId][ticker] + quantity;
  historicalHoldings[walletId].USDT = historicalHoldings[walletId].USDT - usdValue;
}
const userId = info.userId;

// 7D XP uses number of trades, so increment once per trade row.
tradeCountByUser[userId] = (Number(tradeCountByUser[userId]) || 0) + 1;
}

// Count how many wallets each user created in the last 7 days.
const walletCreated7dByUser: Record<string, number> = {};
for (let i = 0; i < wallets.length; i += 1) {
  const wallet = wallets[i];
  const createdMs = new Date(String(wallet.createddate)).getTime();
  if (createdMs < cutoffMs) continue;

  const userId = key(wallet.user_id);
  walletCreated7dByUser[userId] = (Number(walletCreated7dByUser[userId]) || 0) + 1;
}

// Stake totals are tracked separately from wallet assets.
// We store a current and historical value per wallet.
const currentStakeValueByWallet: Record<string, number> = {};
const historicalStakeValueByWallet: Record<string, number> = {};

for (let i = 0; i < stakes.length; i += 1) {
  const stake = stakes[i];
  const walletId = String(stake.walletid);

  // Only active stakes should contribute to current/historical portfolio value.
  const status = String(stake.status || '').toLowerCase();
  if (status !== 'active') {
    continue;
  }

  // Stake value = staked principal + rewards accrued.
  const stakeValue = (Number(stake.quantity) || 0) + (Number(stake.rewardsaccrued) || 0);

  // Current stake value for this wallet.
  currentStakeValueByWallet[walletId] = (Number(currentStakeValueByWallet[walletId]) || 0) + stakeValue;

  // If stake already existed before cutoff, include it in historical stake value.

```

```

    if (new Date(String(stake.createdat)).getTime() <= cutoffMs) {
        historicalStakeValueByWallet[walletId] = (Number(historicalStakeValueByWallet[walletId]) || 0) +
        stakeValue;
    }
}

// Price caches:
// - currentPrices[ticker] is live price from Binance ticker endpoint.
// - historicalPrices[ticker] is close price near cutoff from Binance klines.
// Prices are fetched lazily when first needed and then reused.
const currentPrices: Record<string, number> = {};
const historicalPrices: Record<string, number> = {};
// Group wallet-level performance under each user.
const walletScoresByUser: Record<
    string,
    Array<{ walletName: string; portfolioAllTime: number; portfolio7d: number }>
> = {};
// For each wallet:
// 1) value current holdings
// 2) value historical holdings
// 3) add stake values
// 4) compute all-time and 7d portfolio scores
for (const walletId in walletInfo) {
    const info = walletInfo[walletId];
    let currentAssetsValue = 0;
    const current = currentHoldings[walletId] || {};
    for (const ticker in current) {
        const qty = Number(current[ticker]);
        if (qty === 0) {
            continue;
        }
        // Add any USDT value.
        if (ticker === 'USDT'){
            currentAssetsValue += qty;
        } else {
            // First time we see this ticker, fetch and cache current price.
            if (currentPrices[ticker] === undefined) {
                const pair = ticker + 'USDT'
                try {
                    const res = await fetch(`https://api.binance.com/api/v3/ticker/price?symbol=${pair}`);

                    const data = await res.json();
                    currentPrices[ticker] = Number((data as { price?: unknown })?.price) || 0;
                } catch {
                    currentPrices[ticker] = 0;
                }
            }
            currentAssetsValue += qty * (Number(currentPrices[ticker]) || 0);
        }
    }
}
//Repeat for historical
let historicalAssetsValue = 0;
const historical = historicalHoldings[walletId] || {};
for (const ticker in historical) {
    const qty = Number(historical[ticker])

    if (ticker === 'USDT'){
        historicalAssetsValue += qty;
    } else {
        // First time we see this ticker historically, fetch and cache cutoff close.
        if (historicalPrices[ticker] === undefined) {
            const pair = ticker.endsWith('USDT') ? ticker : `${ticker}USDT`;
            try {
                const res = await fetch(
                    `https://api.binance.com/api/v3/klines?symbol=${pair}&interval=1d&limit=1&endTime=${cutoffMs}`,
                );
                const data = await res.json();
                historicalPrices[ticker] = Number((data as unknown[])?.[0] && (data as unknown[][])[0]?.[4]) || 0;
            } catch {
                historicalPrices[ticker] = 0;
            }
        }
    }
}

```

```

    }
    historicalAssetsValue += qty * (historicalPrices[ticker]);
  }
}

// Add stake value to each side before computing portfolio metrics.
const currentStakeValue = Number(currentStakeValueByWallet[walletId]) || 0;
const historicalStakeValue = Number(historicalStakeValueByWallet[walletId]) || 0;

const walletAllTimeValue = currentAssetsValue + currentStakeValue;
const walletHistoricalValue = historicalAssetsValue + historicalStakeValue;

// All-time compares against the wallet baseline.
// 7d compares current value vs estimated value at cutoff.
const portfolioAllTime = walletAllTimeValue - WALLET_STARTING_USDT;
const portfolio7d = walletAllTimeValue - walletHistoricalValue;

if (!walletScoresByUser[info.userId]) {
  walletScoresByUser[info.userId] = [];
}

walletScoresByUser[info.userId].push({
  walletName: info.walletName,
  portfolioAllTime,
  portfolio7d,
});
}

// Build one final leaderboard payload row per user.
const result: LeaderboardUser[] = [];
for (let i = 0; i < users.length; i += 1) {
  const user = users[i];
  const userId = key(user.id);
  const userWallets = walletScoresByUser[userId] || [];

  let bestAllTimeScore = 0;
  let best7dScore = 0;
  let bestAllTimeWalletName = '';
  let best7dWalletName = '';

  for (let j = 0; j < userWallets.length; j += 1) {
    const wallet = userWallets[j];
    const allTime = Number(wallet.portfolioAllTime) || 0;
    const sevenDay = Number(wallet.portfolio7d) || 0;

    if (j === 0 || allTime > bestAllTimeScore) {
      bestAllTimeScore = allTime;
      bestAllTimeWalletName = String(wallet.walletName || '');
    }

    if (j === 0 || sevenDay > best7dScore) {
      best7dScore = sevenDay;
      best7dWalletName = String(wallet.walletName || '');
    }
  }

  // 7d XP combines activity counts from trades and wallet creation.
  const trades7d = Number(tradeCountByUser[userId]) || 0;
  const wallets7d = Number(walletCreated7dByUser[userId]) || 0;
  const xp7d = trades7d * XP_PER_TRADE + wallets7d * XP_PER_WALLET;

  const username = user.username;

  result.push({
    userId,
    username: typeof username === 'string' ? username : username == null ? null : String(username),
    email: typeof user.email === 'string' ? user.email : user.email == null ? null : String(user.email),
    xpAllTime: typeof user.xp === 'number' ? user.xp : user.xp == null ? null : Number(user.xp) || 0,
    xp7d,
    portfolioAllTime: bestAllTimeScore,
    portfolio7d: best7dScore,
    portfolioAllTimeWalletName: bestAllTimeWalletName,
  });
}

```

```
        portfolio7dWalletName: best7dWalletName,  
    });  
}  
return result;  
}
```

FILE: src/utils/onboardcheck.ts

```
import { createClient, type User } from '@supabase/supabase-js'

const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL!,
  import.meta.env.VITE_SUPABASE_ANON_KEY!
);

export default async function onboardCheck(user: User | null): Promise<boolean> {
  if (!user) return false;

  const { data, error } = await supabase
    .from('tblusers')
    .select('onboarded')
    .eq('id', user.id)
    .single();

  if (error) {
    console.error('Error checking onboarding status:', error);
    return false;
  }

  return data.onboarded;
}
```

FILE: src/utils/stakePageUtils.ts

```
import type { User } from '@supabase/supabase-js';
import type { wallet } from '../types/Wallet';
import fetchWalletUSDTBalance from './FetchWalletUSDTBalance';

export const fetchStakeWalletData = async (user: User | null, preferredWalletId = '') => {
  if (!user) {
    return {
      nextWallets: [] as wallet[],
      selectedWalletId: '',
    };
  }

  const nextWallets = await fetchWalletUSDTBalance(user);

  let selectedWalletId = '';
  if (preferredWalletId) {
    for (const walletItem of nextWallets) {
      if (String(walletItem.walletid) === preferredWalletId) {
        selectedWalletId = preferredWalletId;
      }
    }
  }

  if (selectedWalletId === '' && nextWallets.length > 0) {
    selectedWalletId = String(nextWallets[0].walletid);
  }

  return { nextWallets, selectedWalletId };
};
```

FILE: src/utils/userBadges.ts

```
import type { User } from '@supabase/supabase-js';
import fetchXp from './fetchXP';
import fetchWallets from './FetchWallets';
import { supabase } from '../lib/supabaseClient';
import { badgeDefinitions } from '../data/BadgeDefinitions';

export default async function fetchUserBadgeProgress(user: User){
  //Fetch statistics
  const xp = await fetchXp(user);
  const wallets = await fetchWallets(user);

  let tradeCount = 0;
  for (const wallet of wallets) {
    //Validate wallet id
    if (!wallet.walletid) {
      continue;
    }
    const walletId = wallet.walletid
    const { count, error } = await supabase
      .from('tbltrades')
      .select('tradeid', { head: true, count: 'exact' })
      .eq('walletid', walletId);
    if (error) {
      console.error(`Error fetching trade count for wallet ${walletId}:`, error);
      continue;
    }
    tradeCount += count!;
  }
  const badges = []
  for (const badge of badgeDefinitions) {
    const unlocked = badge.isUnlocked(xp, tradeCount);

    badges.push({
      id: badge.id,
      symbol: badge.symbol,
      title: badge.title,
      description: badge.description,
      unlocked,
    });
  }
  return {
    badges,
    xp,
    tradeCount,
  };
}
```